

无刷直流电动机控制变得容易

Author: Ward Brown
Microchip Technology Inc.

介绍

该应用说明讨论了为无刷电动机开发多个控制器的步骤。覆盖有传感，无传感器，开环和闭环设计。甚至还有一个具有独立电压和速度控制的控制器，因此您可以从经验上发现电动机的特征。

此应用程序注释中的代码是使用Microchip PIC16F877PIC®微控制器与电路调试器（ICD）一起开发的。之所以选择这种组合，是因为ICD很便宜，并且可以在原型硬件中调试代码，而无需额外的程序员或仿真器。随着设计的开发，我们将焦油设备编程并直接从

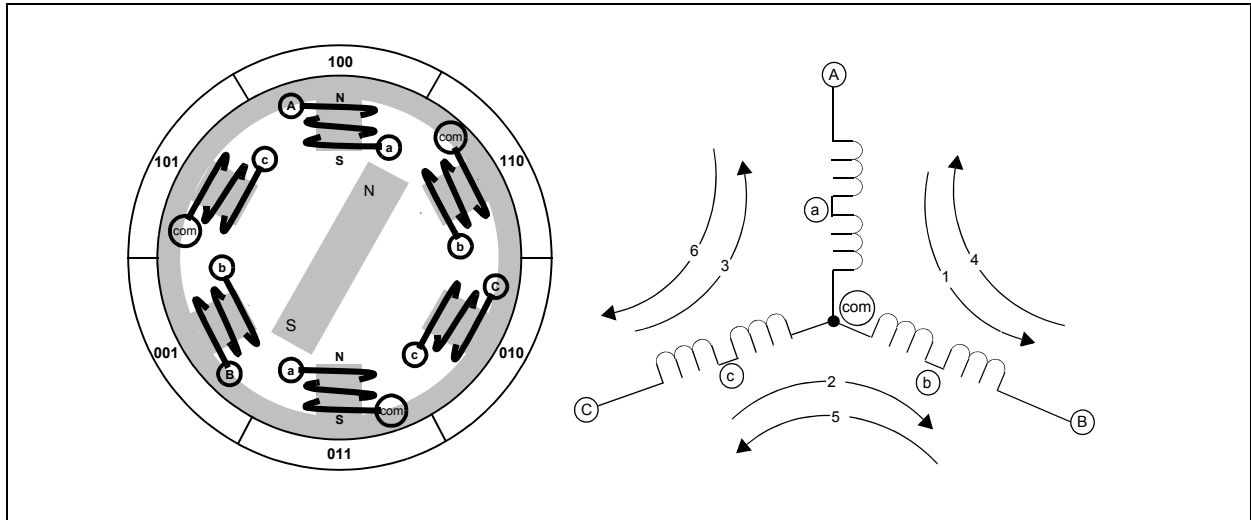
Mplab®环境。然后，最终代码可以移植到较小，较便宜的PIC微控制器之一。移植需要最小的努力，因为所有PIC 14位核心设备的说明集都是相同的。

还应注意，该代码对Pittman N2311A011无刷直流电机进行了测试和优化。还测试了其他电动机，以确保该代码通常很有用。

BLDC的解剖学

图1是BLDC电动机结构的简化说明。无刷电动机是用固定的磁铁转子和电线伤口定子杆构造的。电能通过永久磁铁转子和伤口定子杆中诱导的旋转磁场之间的磁力转化为机械能。

FIGURE 1: SIMPLIFIED BLDC MOTOR DIAGRAMS



在此示例中，有三个电磁电路在一个共同点连接。每个电磁电路都在中心分开，从而使磁转子在诱导磁场的中间移动。大多数BLDC电动机具有带有星形连接的三相绕组拓扑。带有此拓扑的电动机一次通过一次启动两个阶段来驱动。图2中所示的静态对齐是通过创建从端子A到B的电流流量来实现的，该电流在图1中的示意图上指出为路径1。可以使转子通过将电流路径从端子C到B的流动为B上，将60度旋转为60度从A到B对准，该路径从端子C到B，在原理上旋转为B。Sug-Gest Gest的磁对齐仅用于说明目的，因为它易于可视化。实际上，当永久性磁芯距离与定子磁场对齐90度时，将获得最大扭矩。

BLDC换向的关键是感知转子位置，然后为产生最多扭矩的相位能量。转子每个换向步骤都会行驶60个电度。当转子与相应的定子磁场对齐为120度时，将适当的定子电流路径激活，然后当转子与比60度时停用，然后将下一个电路作用并重复过程。图1所示的转子位置的换向位置将在当前路径2的完整性和顺时针旋转的当前路径3的开始。通勤电气

通过六个可能的组合进行CAL连接，编号为1到6，在正确的时刻，将在一场电革命中拉动转子。

在图1的简化电动机中，一个电撤销与一场机械革命相同。在实际实践中，BLDC电动机具有以上显示的电路之一，彼此并行连接，以及相应的多孔永久性磁盘。对于两个电路，有两个机械革命的电气转换，因此，对于两回路电动机，每个电换向相都将覆盖30度的机械旋转。

感应的换向

了解绕线电流的正确时刻的最简单方法是通过位置传感器。许多BLDC电动机制造商为电动机提供了三元素效果位置传感器。每个传感器元件以180个电度的电旋转为数字高水平，而其他180个电度则低水平。这三个传感器相互偏移60个电度，以使每个传感器输出与电磁电路之一对齐。图2显示了显示传感器输出与所需电动机驱动电压之间关系的定时图。

FIGURE 2: SENSOR VERSUS DRIVE TIMING

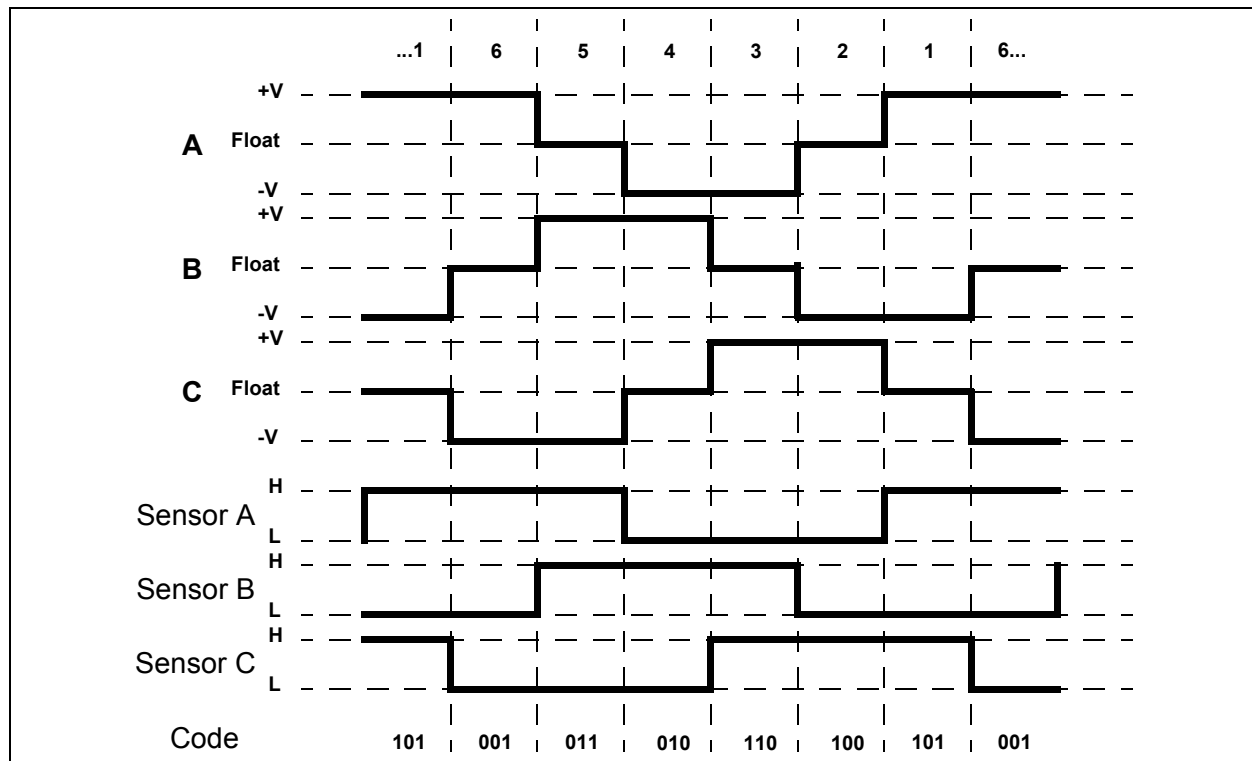


图2的顶部的数字对应于图1中所示的当前阶段。从图2中可以明显看出，三个传感器输出以创建六个独特的三位数代码的方式重叠。图1中电机图周围显示的数字表示传感器位置代码。转子的北极指向在转子位置输出的代码。数字是传感器逻辑级别，其中最显着的位是传感器C，而最小的位是传感器A。

每个驱动阶段由一个电动机端子驱动高，一个电动机端子驱动的低和一个电动机左漂浮。简化的驱动电路如图3所示。高和低驱动器的单个驱动器控件允许在每个电动机端子上进行高驱动，低驱动器和浮动驱动器。必须使用这种类型的驾驶员电路采取的一种预防措施是，不得同时激活高侧和低侧驱动器。必须将上拉电阻和下拉电阻放在驱动器输入上，以确保在微控制器重置后立即关闭驱动程序，当将微控制器输出配置为高阻滞的输入时。

同时针对两位驾驶员活跃的另一个预防措施称为“死时间控制”。当输出从高驱动器状态到低驱动器状态的过渡时，必须允许将高侧驱动器关闭的适当时间在激活低侧驱动器之前通过。驾驶员要比打开需要更多的时间关闭，因此必须允许额外的时间经过，以使两个驾驶员都没有同时进行。注意图3中，每个输出的高驱动周期和低驱动周期都通过浮动驱动阶段分开。这个死时间是三相BLDC驱动方案所固有的，因此不需要特殊的时间控制。 bldc

换向序列将永远不会同时切换高侧设备和低侧设备。

在这一点上，我们准备开始构建电动机换向控制代码。换向包括将输入传感器状态与相应驱动状态链接起来。最好使用状态表和表偏移指针来实现这一点。传感器输入将形成表偏移指针，并且可能的输出驱动器代码的列表将形成状态表。代码开发将使用ICD中的PIC16F877执行。PORTC已任意分配为电机驱动端口和Porte作为传感器输入端口。选择PORTC作为驱动器端口，因为ICD演示板上还在该端口上有LED指标，因此我们可以在没有任何外部测试设备的情况下观看慢速换向驱动器信号。

每个驾驶员需要两个销钉，一个用于高驱动器，一个用于低速驱动器，因此将使用六个PORTC来控制六个电动机驱动器MOSFET。每个传感器需要一个销钉，因此将使用三个PORTE来读取电动机三输出传感器的当前状态。传感器状态将通过使用传感器输入代码作为驱动器表索引的二进制偏移来链接到驱动状态。图2中的传感器状态和运动驱动状态在表1中列出。

图3：三相桥

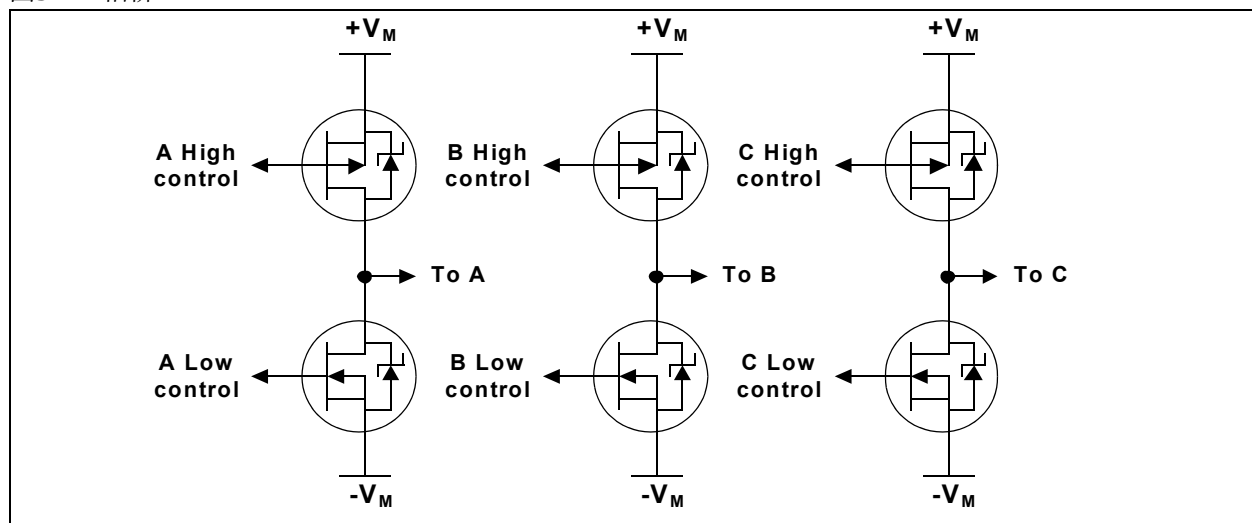


表1: CW传感器和驱动器按相顺序

Pin	RE2	RE1	RE0	RC5	RC4	RC3	RC2	RC1	RC0
Phase	Sensor C	Sensor B	Sensor A	C High Drive	C Low Drive	B High Drive	B Low Drive	A High Drive	A Low Drive
1	1	0	1	0	0	0	1	1	0
2	1	0	0	1	0	0	1	0	0
3	1	1	0	1	0	0	0	0	1
4	0	1	0	0	0	1	0	0	1
5	0	1	1	0	1	1	0	0	0
6	0	0	1	0	1	0	0	1	0

表2中的传感器代码二进制重量的排序表1。根据表2构建的状态表激活电动机，将导致图1的电动机顺时针旋转。

表2: CW传感器和驱动器按传感器顺序

Pin	RE2	RE1	RE0	RC5	RC4	RC3	RC2	RC1	RC0
Phase	Sensor C	Sensor B	Sensor A	C High Drive	C Low Drive	B High Drive	B Low Drive	A High Drive	A Low Drive
6	0	0	1	0	1	0	0	1	0
4	0	1	0	0	0	1	0	0	1
5	0	1	1	0	1	1	0	0	0
2	1	0	0	1	0	0	1	0	0
1	1	0	1	0	0	0	1	1	0
3	1	1	0	1	0	0	0	0	1

逆时针旋转是通过沿与该方向相反的方向穿过电动机线圈以进行顺时针旋转来实现的。表3是通过交换表2的所有高和低驱动器来构建的。根据表3构建的状态表，激活电动机线圈将导致电动机逆时针旋转。表3中的相数是斜线，表示EMF与表2中的相相反。

表3: CCW传感器和驱动器位

Pin	RE2	RE1	RE0	RC5	RC4	RC3	RC2	RC1	RC0
Phase	Sensor C	Sensor B	Sensor A	C High Drive	C Low Drive	B High Drive	B Low Drive	A High Drive	A Low Drive
/6	0	0	1	1	0	0	0	0	1
/4	0	1	0	0	0	0	1	1	0
/5	0	1	1	1	0	0	1	0	0
/2	1	0	0	0	1	1	0	0	0
/1	1	0	1	0	0	1	0	0	1
/3	1	1	0	0	1	0	0	1	0

用于从传感器输入确定适当驱动词的代码段如图4所示。

FIGURE 4: COMMUTATION CODE SEGMENT

```

#define DrivePort      PORTC
#define SensorMask     B'00000111'
#define SensorPort     PORTE
#define DirectionBit   PORTA, 1

Commutate
    movlw  SensorMask      ;retain only the sensor bits
    andwf  SensorPort      ;get sensor data
    xorwf  LastSensor, w   ;test if motion sensed
    btfsc  STATUS, Z       ;zero if no change
    return                ;no change - return

    xorwf  LastSensor, f   ;replace last sensor data with current
    btfss  DirectionBit    ;test direction bit
    goto   FwdCom          ;bit is zero - do forward commutation

    ;reverse commutation
    movlw  HIGH RevTable   ;get MS byte to table
    movwf  PCLATH          ;prepare for computed GOTO
    movlw  LOW RevTable    ;get LS byte of table
    goto   Com2

FwdCom
    ;forward commutation
    movlw  HIGH FwdTable   ;get MS byte of table
    movwf  PCLATH          ;prepare for computed GOTO
    movlw  LOW FwdTable    ;get LS byte of table

Com2
    addwf  LastSensor, w   ;add sensor offset
    btfsc  STATUS, C       ;page change in table?
    incf   PCLATH, f       ;yes - adjust MS byte

    call   GetDrive        ;get drive word from table
    movwf  DriveWord       ;save as current drive word
    return

GetDrive
    movwf  PCL

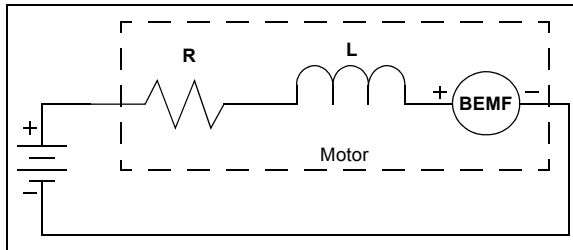
FwdTable
    retlw  B'00000000'     ;invalid
    retlw  B'00010010'     ;phase 6
    retlw  B'00001001'     ;phase 4
    retlw  B'00011000'     ;phase 5
    retlw  B'00100100'     ;phase 2
    retlw  B'00000110'     ;phase 1
    retlw  B'00100001'     ;phase 3
    retlw  B'00000000'     ;invalid

RevTable
    retlw  B'00000000'     ;invalid
    retlw  B'00100001'     ;phase /6
    retlw  B'00000110'     ;phase /4
    retlw  B'00100100'     ;phase /5
    retlw  B'00011000'     ;phase /2
    retlw  B'00001001'     ;phase /1
    retlw  B'00010010'     ;phase /3
    retlw  B'00000000'     ;invalid

```

在使用电动机尝试换向代码之前，让我们考虑将电压应用于直流电动机时会发生什么。DC电动机的大量简化电气模型如图5所示。

FIGURE 5: DC MOTOR EQUIVALENT CIRCUIT



当转子静止时，唯一的抗流流量是电磁线圈的阻抗。阻抗由绕组中铜的寄生抗性以及绕组本身的寄生诱导。设计的电阻和电感非常小，因此启动术的车程将非常大，即使不是限制。

当电动机旋转时，永久磁铁转子经过定子线圈会诱导称为后电力或BEMF的线圈中的电势。BEMF与电动机速度成正比，并根据电动电压构为KV确定。

等式1:

$$\begin{aligned} \text{RPM} &= K_V \times \text{Volts} \\ \text{BEMF} &= \text{RPM} / K_V \end{aligned}$$

在理想的电动机中，R和L为零，电动机将以速率旋转，以使BEMF完全等于施加的电压。

电动机绘制的电流与电动机轴上的扭矩负载成正比。电机电流由电动机扭矩常数KT确定。

等式2:

$$\text{扭矩} = k_t \times \text{amps}$$

关于KT和KV的一个有趣的事实是，所有电动机的产品都是相同的。Volt和Amps以MKS单位表示，因此，如果我们还以N-M/rad/sec (n-m rad/sec) 表示KT，则KV和KT的乘积为1。

等式3:

$$K_V \times K_T = 1$$

当您认为产品的单位为 $[1/(\text{v} \cdot \text{a})] \cdot [(\text{n} \cdot \text{m}) \cdot (\text{rad}/\text{sec})]$ 时，这并不奇怪，这与由电力划分的机械功率相同。

如果要从理想的电压源将电压应用于理想电动机，它将绘制无限量的电流，并立即加速到由施加的电压和KV所决定的速度。当然，没有电动机是理想的，启动电流将受到电动机绕组的抗性和电感的限制，以及电源的电流容量。无限制的启动电流和电压的两种有害影响是过度的扭矩和过度的截面。过度的扭矩会导致齿轮剥离，轴耦合滑动以及其他不良的机械问题。过多的电流会导致驾驶员MOSFET吹出并电路燃烧。

我们可以通过使用脉冲宽度调制（PWM）限制启动时施加的电压来最大程度地减少过度电流和扭矩的影响。脉冲宽度调节是有效的，而且非常简单。PWM需要考虑的两件事是，由于切换而导致的MOSFET损失以及PWM速率对电动机的影响。较高的PWM频率意味着更高的开关损耗，但是PWM频率太低将意味着电动机的电流将是一系列高电流脉冲，而不是电压波形的所需平均值。如果寄生运动电感相对较高，则在较低的频率下更容易达到平均，但是高电感是一种不受欢迎的运动特征。理想频率取决于电动机和电动开关的特性。对于此应用，PWM频率将大约为10 kHz。

我们正在使用PWM来控制启动电流，那么为什么不将其用作速度控制呢？我们将使用PIC16F877的Analog-Digital转换器（ADC）读取电位计，并将电压读数用作相对速度控制输入。仅使用8位ADC，因此我们的速度控制将具有256个级别。我们希望相对速度与相对电位计位置相对应。电动机速度直接受到施加电压的影响，因此将PWM占空比从0%变化到100%，将导致线性速度控制从最大rpm的0%到100%。脉冲宽度是通过将ADC结果连续添加到自由运行timer0计数中以确定驾驶员应打开或关闭时确定的。如果增加导致溢出，则驱动程序将打开，否则他们会关闭。使用8位计时器，以便ADC到计时器添加不需要缩放来覆盖整个范围。要获得10 kHz的PWM频率，必须以256倍的速率运行，或2.56 MHz。Timer0的最小预定值为1: 2，因此我们需要5.12 MHz的输入频率。timer0的输入是fosc/4。这需要20.48 MHz的FOSC。这是一个奇怪的频率，20 MHz足够接近，因此我们将使用20 MHz，导致PWM频率为9.77 kHz。

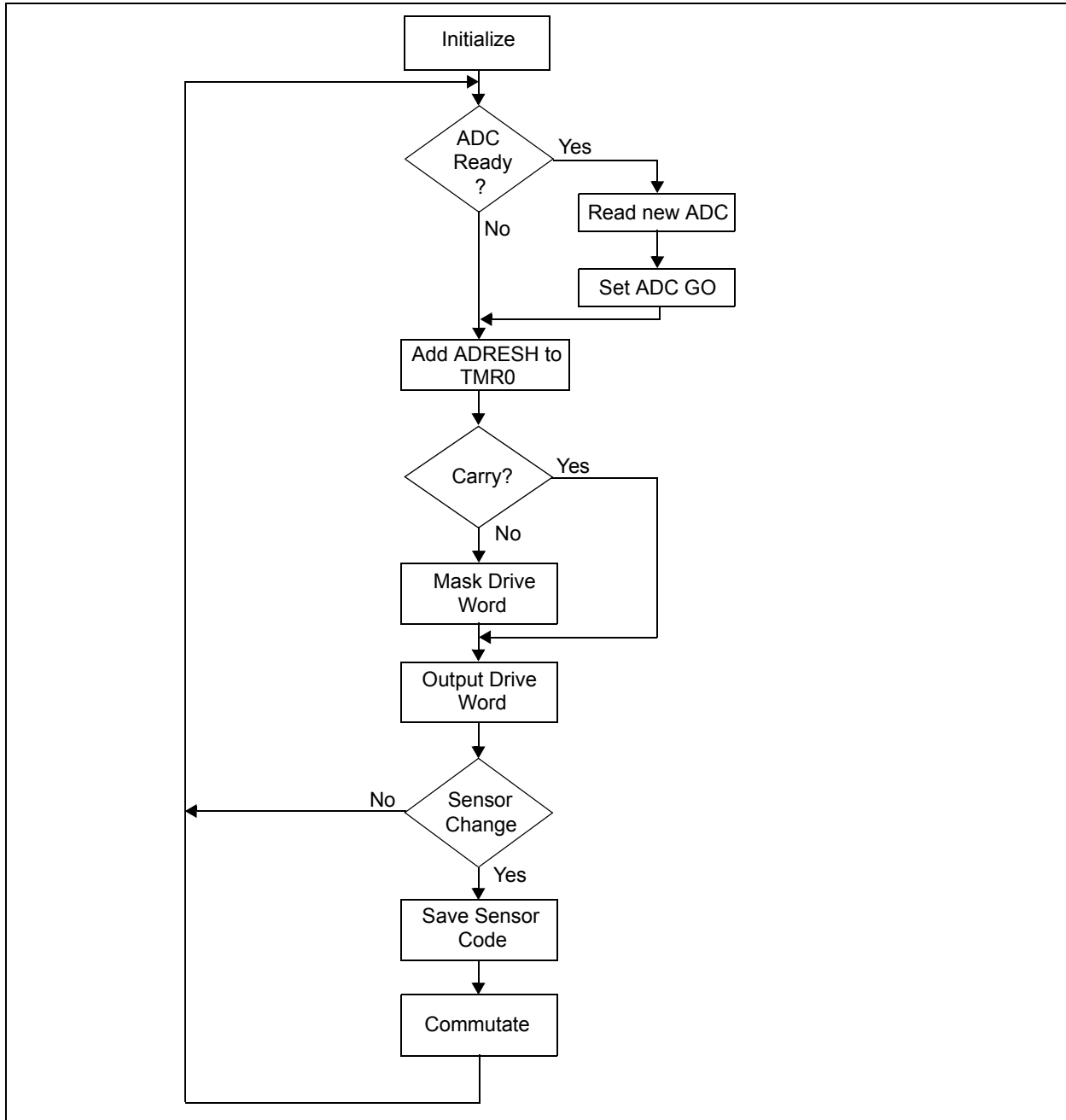
有几种调节电动机驱动器的方法。我们可以将高和低侧驱动器切换在一起，或者只是在离开另一个驾驶员时，只有高驾驶员或低驾驶员。一些高侧MOSFET驾驶员使用电容器充电泵来增加闸门驱动器上方。当驾驶员关闭并在驾驶员打开时将充电泵充电并排出MOSFET门。然后，切换高侧驱动器以保持充电泵的刷新是有道理的。即使此应用程序不使用电荷泵类型驱动程序，我们也会在离开低侧驱动器时调制高侧驱动器。有三个高侧驱动器，其中任何一个都可以根据转子的位置而活跃。电动机驱动词宽6位，因此，如果我们在逻辑上和高驾驶员位置的零驱动词和零位置，而在低驾驶员位置位置为1，则无论三个是哪一个。

现在，我们已经确定了控制循环的4个任务：

- 阅读传感器输入
- 交换电动机连接
- 阅读速度控制ADC
- PWM使用ADC和Timer0添加结果的电动机驱动器

以20 MHz的时钟速率，由循环时间引起的控制延迟并不重要，因此我们将构建一个简单的投票任务循环。控制循环流程图如图6所示，代码列表在附录B中。

FIGURE 6: SENSORED DRIVE FLOWCHART



无传感器电机控制

可以通过在其中一个驱动器阶段在未发达的电动机端子上感测后电动势电压来确定何时将电动机驱动电压接线。无传感器控制的明显成本优势是消除了霍尔位置传感器。无传感器控制有几个缺点：

- 电动机必须以最低速率移动才能产生足够的背部电动势以感知
- 电动机负载突然更改会导致BEMF驱动环脱离锁
- 只有当电动机速度在施加电压的理想换向率范围内，才能测量BEMF电压
- 比理想速度快的换向速度将导致不连续的电机响应

如果低成本是主要问题，并且低速电动机的操作不是必需的，并且预计电动机负载不会迅速变化，那么无传感器控制可能是您应用的更好选择。

确定BEMF

相对于线圈公共连接点，由每个电动机线圈生成的BEMF可以表示如等式4至等式6所示。

等式4：

$$B_{BEMF} = \sin(\alpha)$$

等式5：

$$C_{BEMF} = \sin\left(\alpha - \frac{2\pi}{3}\right)$$

等式6：

$$A_{BEMF} = \sin\left(\alpha - \frac{4\pi}{3}\right)$$

FIGURE 7: BEMF EQUIVALENT CIRCUIT

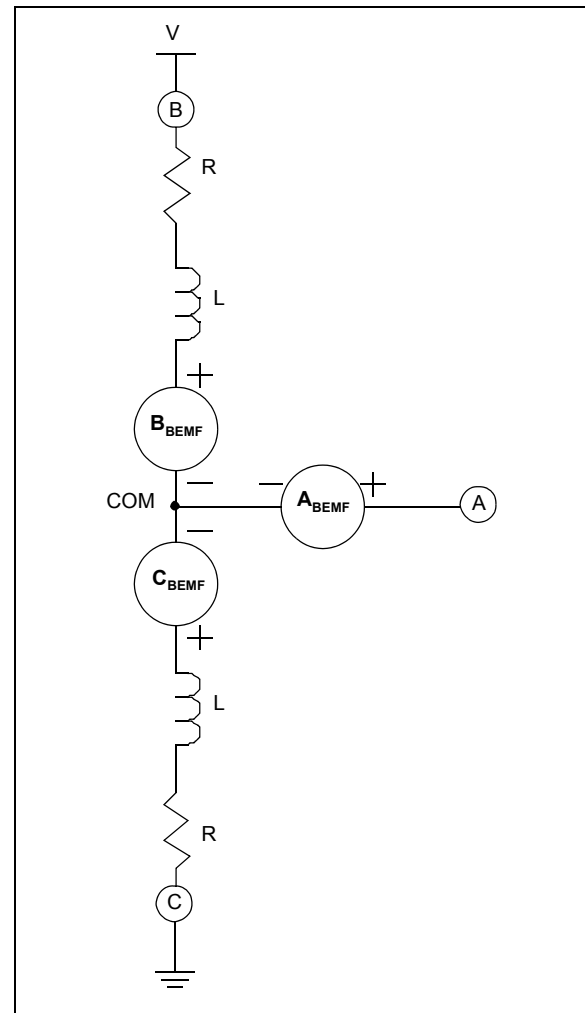


图7显示了电动机与线圈B和C驱动的等效电路，而线圈A是未驱动的，可用于BEMF测量。在换向的频率上，L的可忽略不计。假定R相等。L和R组件在A分支中未显示，因为电路的这一部分中没有明显的电流流，因此可以忽略这些组件。

如图7所示，由B和C线圈产生的BEMF可以表示如图7所示。

等式7:

$$BEMF_{BC} = B_{BEMF} - C_{BEMF}$$

C_{BEMF} 的符号逆转是由于将参考点从共同连接移到地面上。

回想一下，一场电气革命有六个驱动器阶段。每个驱动阶段发生+/- 30度，围绕在该阶段驱动的两个电动机绕组的峰后电动势。在全速下，所施加的直流电压等效于60度范围内的RMS BEMF电压。就任何一个绕组所产生的峰值BEMF而言，跨两个绕组的RMS BEMF电压可以表示如等式8所示。

等式8:

$$BEMF_{RMS} = \sqrt{\frac{3}{\pi} \int_{\frac{\pi}{6}}^{\frac{\pi}{2}} \left(\sin(\alpha) - \sin\left(\alpha - \frac{2\pi}{3}\right) \right)^2 d\alpha}$$

$$BEMF_{RMS} = \sqrt{\frac{3}{\pi} \left(\frac{\pi}{2} + \frac{3\pi}{4} \right)}$$

$$BEMF_{RMS} = 1.6554$$

我们将使用此结果将稍后介绍的BEMF图归一化，但首先让我们考虑在未发达的运动端子处的预期BEMF。

由于施加的电压是脉冲宽度调制的，因此在整个阶段时间内和关闭之间的驱动器交替。相对于地面，在驱动器打开时在A端子上看到的BEMF可以如公式9所示表示。

等式9:

$$BEMF_A = \frac{[V - (B_{BEMF} - C_{BEMF})]R}{2R} - C_{BEMF} + A_{BEMF}$$

$$BEMF_A = \frac{V - B_{BEMF} + C_{BEMF}}{2} - C_{BEMF} + A_{BEMF}$$

请注意，绕组电阻会取消，因此在测量BEMF时，由于电动机扭矩负载而导致的电阻降低并不是一个因素。

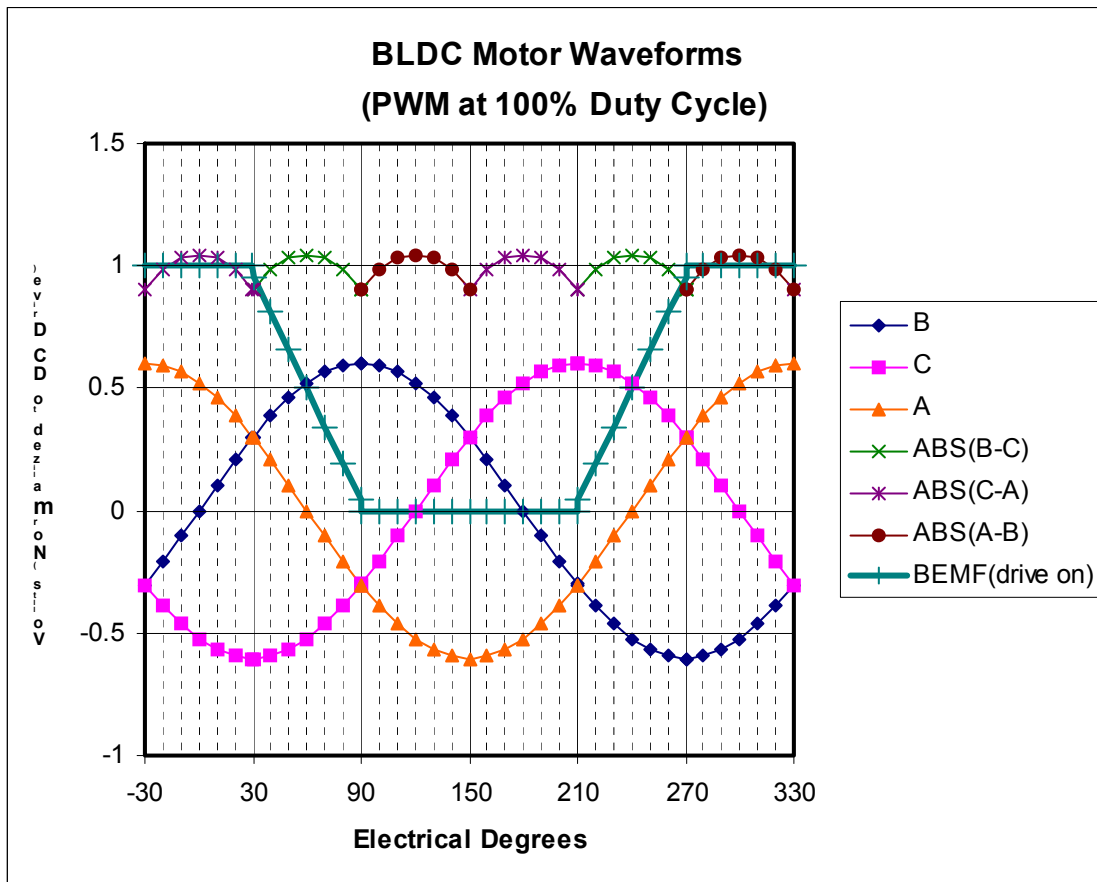
当驱动器关闭时，在A端子上看到的BEMF相对于地面，如公式10所示。

等式10:

$$BEMF_A = A_{BEMF} - C_{BEMF}$$

图8是在一场电革命中计算出的BEMF FOR-MULA的图形表示。为了避免混乱，只有在示波器上观察到的，仅显示波形，并将其表示为BEMF（开机）。端子A波形在顶部和底部扁平，因为在那个点，端子连接到驱动电压或接地。正弦波形是相对于线圈公共连接点的个体线圈BEMF。60度正弦驼峰是相对于地面的驱动线圈对的BEMF。整个图已归一化为线圈对BEMF的RMS值。

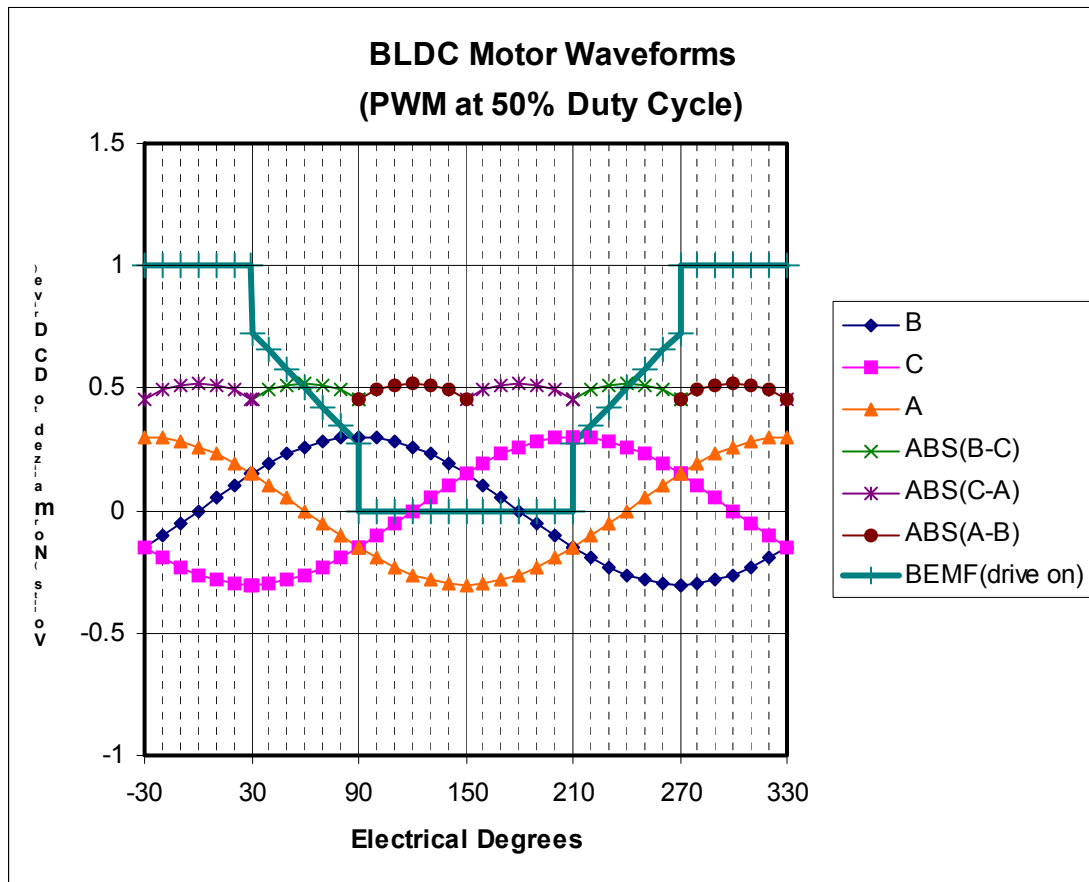
FIGURE 8: BEMF AT 100% DRIVE



请注意，BEMF（驱动）波形是相当线性的，并且通过正好是精确60度的施加电压的一半的电压，该电压与线圈A BEMF波形的零交叉相连。这意味着我们可以通过检测开放式电压等于施加电压的一半来确定转子电位。

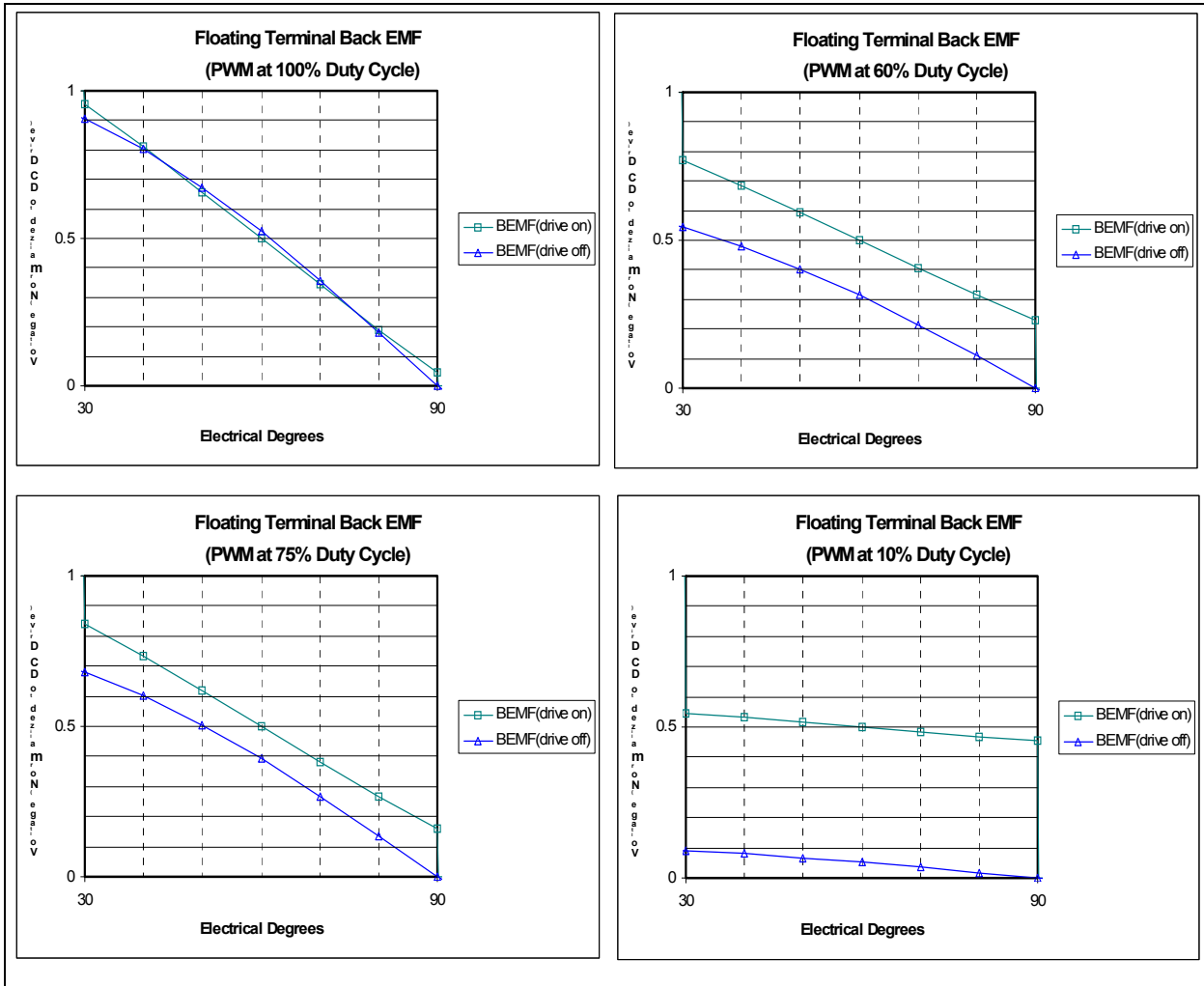
当PWM占空比小于100%时会发生什么？图9是当有效的施加电压是图8中所示的50%时，在一个电旋转上计算出的BEMF公式的图形表示。整个图已正常为峰施加电压。

FIGURE 9: BEMF AT 50% DRIVE



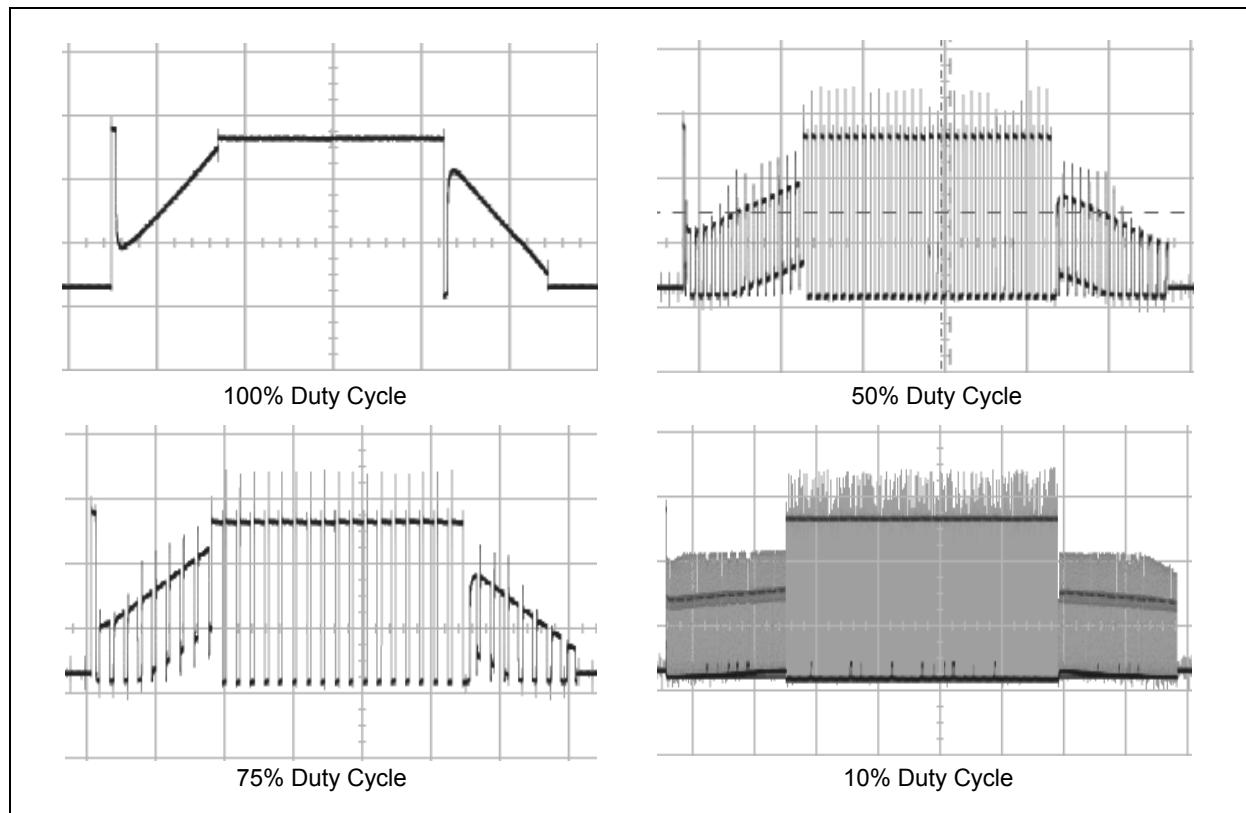
如预期的那样，BEMF波形全部降低，但请注意，开放式末端上的BEMF仍等于在60度驱动阶段中途施加的电压的一半。仅当驱动电压打开时才发生。图10显示了驱动电压打开以及驱动电压熄灭时的开放端子BEMF的细节。在各种占空比的周期中，请注意，曲线上的驱动始终等于施加的电压的一半在60度。

FIGURE 10: DRIVE ON VS. DRIVE OFF BEMF



这些预测与实际电动机的符合程度如何？图11显示了Pittman N2311A011无刷电动机的端子A上存在的波形。大型瞬变，尤其是在100%占空比波形中普遍存在的，是由于电动电感引起的反射电流。

FIGURE 11: PITTMAN BEMF WAVEFORMS



当应用驱动电压时，可以通过测量开放端子上的电压来确定转子位置，然后将结果与施加电压的一半进行比较。

回想一下，电动机速度与施加的电压成正比。当换政率与有效施加电压共同进行时，公式和图表呈现的电动机运行。当换向率太快时，换向就发生了，零交叉点发生在驱动阶段后期。当换向率太慢时，换向的时间很晚，零交叉点发生在驱动阶段。我们可以感知并利用零交叉的这种转移来调整通信速率，以使电动机以理想的速度运行，以实现施加的电压和负载扭矩。

开环速度控制

无刷直流电动机的一个有趣属性是，它们将在一定程度上同步操作。这意味着，对于给定的负载，施加的电压和组合速率，如果这三个变量不会大量偏离理想，则电动机将保持开放环锁定。理想是由电机电压和扭矩常数确定的。这是如何运作的？考虑到，当换向率太慢而对于应用电压时，BEMF会太低，从而导致更多的电动电流。电动机将通过加速到下一个阶段位置而做出反应，然后放慢速度等待下一个换向。在极端情况下，电动机将像步进电动机一样捕捉到每个位置，直到下一次换向。由于电动机能够加速的速度快于换向率，因此可以容忍速度慢得多，而失去锁定的速度可以容忍，但以过高的电流为代价。

现在，考虑到换向太快时会发生什么。当换向的早期发生时，BEMF尚未达到峰值，导致更多的电机电流和更高的加速度到下一阶段，但它将到达那里太晚。电动机试图跟上换向的能力，但要付出过度的限制。如果换向的时间太早到达，以至于电动机无法足够快地加速以捕获下一个脉冲，那么锁会丢失并且电动机旋转。这种情况突然距离理想速度不远。锁的突然损失看起来像是电动机响应中的不连续性，这使得闭环控制变得困难。闭环控制的一种替代方法是调整组合速率，直到实现自锁开路控制为止。这是我们将在应用中使用的方法。

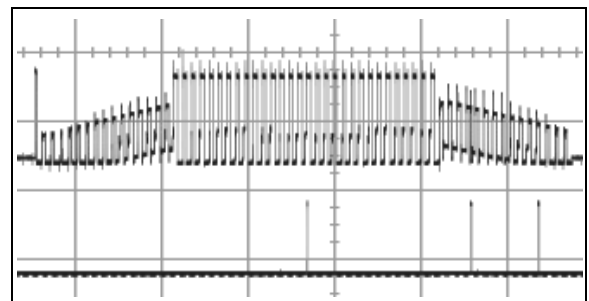
当电动机上的负载在其工作范围内恒定时，则相对于施加电压的电机速度响应曲线是线性的。如果电源电压受到良好的调节，除了恒定的扭矩负载外，则可以在整个速度范围内操作电动机。考虑到脉冲宽度调节，有效电压与PWM占空比成正比。可以通过将PWM占空比将PWM占空比链接到存储的电动机速度表作为每个驱动阶段的换向时间来制作一个开环控制器。我们需要一张桌子，因为每单位时间旋转是线性的，但是我们需要每革命的时间，这不是线性的。查找表中的时间值比反复计算它们要快得多。

我们用来运行电动机打开循环的程序与我们将使用相同的程序来自动调整换向率，以响应扭矩负载的变化。该程序使用两个电位仪作为速度控制输入。我们称其为PWM电位器，一个电位计直接与PWM占空比和换向时间查找表有关。我们称其为偏移电位的第二个电位器用于提供由PWM电位器确定的PWM占空比的偏移。PWM电位器的Ana-log-log-Digital转换可产生0到255之间的数字。通过将PWM电位计读数添加到免费运行的8位计时器中，可以生成PWM占空比。当添加导致随身携带的驱动状态时，否则就会关闭。PWM电位计读数还用于访问256个位置换向时间查找表。偏置电位器还产生了0到255之间的数字。此数字中最重要的位是倒置的，使其在-128至127之间签名。当将其添加到PWM电位计中时，此偏移结果将成为PWM占空比责任循环阈值，并控制驱动器的驱动器和关闭状态所描述的驱动器。

闭环速度控制

闭环速度控制是通过将换政时表索引与PWM占空比数字取消链接的速度控制。PWM电位器将添加到0到255之间的固定手动阈值中。当此添加导致随身携带时，该模式会切换为自动。在输入自动模式时，通勤索引最初设置为PWM电位器读数。此后，只要自动模式仍然有效，则根据在特定时间在电机端子上读取的电压自动调整或向下调整换向索引。进行了三个电压读数。

FIGURE 12: BEMF SAMPLE TIMES



一读是在驱动器第4阶段进行的。这是施加的电压。接下来的两个读数是在驱动器第5阶段浮动时进行的。一读是在过去的通勤时间的 $\frac{1}{4}$ 读数时进行的，并且在通往时间的 $\frac{3}{4}$ 经过时进行二读。我们将分别称为1和2。根据应用电压读数和读数1和2之间的以下关系，对换向表索引进行调整。

- 如果读取1 > 应用伏特/2和读取2 < 应用电压/2，则索引是不变的。
- 如果读取1 < 应用电压/2，则索引将增加
- 如果读取1 > 应用伏特/2和读数2 > 应用电压/2

电机转子及其连接到的一切都具有一定数量的惯性。惯性延迟了电压负载和换向时间变化的电动机响应。延迟到换向时间表索引的更新以补偿机械延迟，并允许电动机赶上。

加速和减速延迟

电动机的惯性及其驱动的趋势倾向于延迟电动机对驱动电压变化的反应。我们需要通过在控制循环中添加匹配延迟来补偿此延迟。控制回路延迟需要两个时间常数，一个相对较慢的加速度和一个相对快速的变速。

考虑到电动机的电压突然上升时，控制循环中会发生什么，或者电动机负载突然降低。控制感知电机旋转太慢，并且试图通过使换向时间缩短来调整。将不延迟控制循环，将进行下一个速度测量

在电动机对调整做出反应之前，将进行另一种速度调整。在电动机响应之前继续进行调整，直到最终，换向时间太短了，对于施加的电压而言，电动机锁定了。加速计时器延迟阻止了这种失控的条件。由于电动机可以忍受太长的换向时间，但不能忍受太短的换向时间，因此加速度延迟的时间延迟可能长于所需的时间，而没有严重的有害效果。

考虑到电动机的电压突然掉落或电动机负载突然增加时，控制循环中会发生什么。如果更改足够大，那么对于电机条件而言，换向时间将立即运行太短。电动机无法忍受这一点，将会发生锁定损失。为了防止锁定的损失，循环减速定时器延迟必须足够短，以使控制循环跟踪或在不断变化的电机状况之前进行跟踪。如果时间延迟太短，则控制循环将继续延长电动机响应之前的换向时间，从而导致过度组件。电动机最终将慢速至BEMF传感器表明速度太慢而无法施加电压的速度。那时，换向减速将停止，换向变化将朝着相反的方向调整，该方向由加速度延迟控制。减速期间的赔偿金不会导致锁定损失，但会导致扭矩波纹和电动电流的水平增加，直到最终达到理想的换向时间为止。

确定换向时间表值

提供MPLAB的汇编器以32位整数的形式执行所有功能。为了避免由整数数学引起的舍入错误，我们将使用电子表格（例如Excel）来计算表条目，然后将结果剪切并粘贴到包含文件中。设置电子表格如表4所示。

表4：换向时间表值

Variable Name	Number or Formula	Description
Phases	12	Number of commutation phase changes in one mechanical revolution.
Fosc	20 MHz	Microcontroller clock frequency
Fosc_4	Fosc/4	Microcontroller timers source clock
Prescale	4	Timer 1 prescale
MaxRPM	8000	Maximum expected speed of the motor at full applied voltage
MinRPM	$(60 * Fosc_4) / Phases * Prescale * 65535 + 1$	Limitation of 16-bit timer
Offset	-345	This is the zero voltage intercept on the RPM axis. A property normalized to the 8-bit A to D converter.
Slope	$(MaxRPM - Offset) / 255$	Slope of the RPM to voltage input response curve normalized to the 8-bit A to D converter.

电子表格的主体在第13行任意启动。第12行包含列标题。电子表格的主体构造如下：

- A列是换向表索引编号N。A列中的数字是整数从0到255。
- B列是通过在索引号N处使用计数器值而产生的RPM。列B中的公式为： $= \text{if}(\text{offset} + \text{a13} * \text{slope} > \text{minrpm}, \text{offset} + \text{a13} * \text{slope}, \text{minrpm})$ 。
- C列是以秒为单位表示的每个换向阶段的持续时间。Column C的公式为： $= 60 / (\text{阶段} * \text{B13})$ 。
- D列是计时器计数中表达的每个换向阶段的持续时间。D列的公式为： $= \text{C13} * \text{FOSC_4} / \text{PRESCALE}$ 。

合理分辨率的换向阶段时间范围需要16位计时器。计时器从0到比较值，然后自动将其重置为0。比较值存储在换向时间表中。由于比较是16位，并且表只能处理8位，因此换向时间将存储在相同索引访问的两个表中。

- E列是16位计时器比较值中最重要的字节。E列的公式为： $= \text{condenate}(\text{"retlw high d'"}, \text{int}(\text{d13}), ', ')$ 。
- F列是16位计时器比较值的最低字节。F列的公式为： $= \text{condenate}(\text{"retlw low d'"}, \text{int}(\text{d13}), ', ')$ 。

当所有电子表格公式都在第13行中输入时，可以将公式拖到第268行以将表扩展到所需的256个条目。columns e和f将以汇编器准备格式的桌子条目。表电子表格的一个示例如图13所示。

图13: PWM查找表生成器

The screenshot shows a spreadsheet titled "BLDC Table Generator.xls". It contains two main sections. The top section is a parameter input table with columns A through F. The bottom section is a lookup table with columns N, RPM, Sec per Transition, Timer Counts, MS Byte Code, and LS Byte Code.

	A	B	C	D	E	F
1	Phases/Rev	12				
2	Fosc	2.00E+07				
3	Fosc/4	5.00E+06				
4	Prescale	4				
5	MaxRPM	8000				
6	MinRPM	96				
7	Offset	-345.00				
8	Slope	32.73				
9						
10						
11						
12	N	RPM	Sec per Transition	Timer Counts	MS Byte Code	LS Byte Code
13	0	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
14	1	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
15	2	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
16	3	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
17	4	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
18	5	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
19	6	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
20	7	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
21	8	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
22	9	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
23	10	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
24	11	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
25	12	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
26	13	96	5.19E-02	64855	retlw high D'64854'	retlw low D'64854'
27	14	113	4.42E-02	55233	retlw high D'55233'	retlw low D'55233'
28	15	146	3.43E-02	42843	retlw high D'42842'	retlw low D'42842'

At the bottom of the spreadsheet, there are navigation buttons and a status bar showing "Lookup Table / Open Loop /".

使用开环控制确定电动机特性

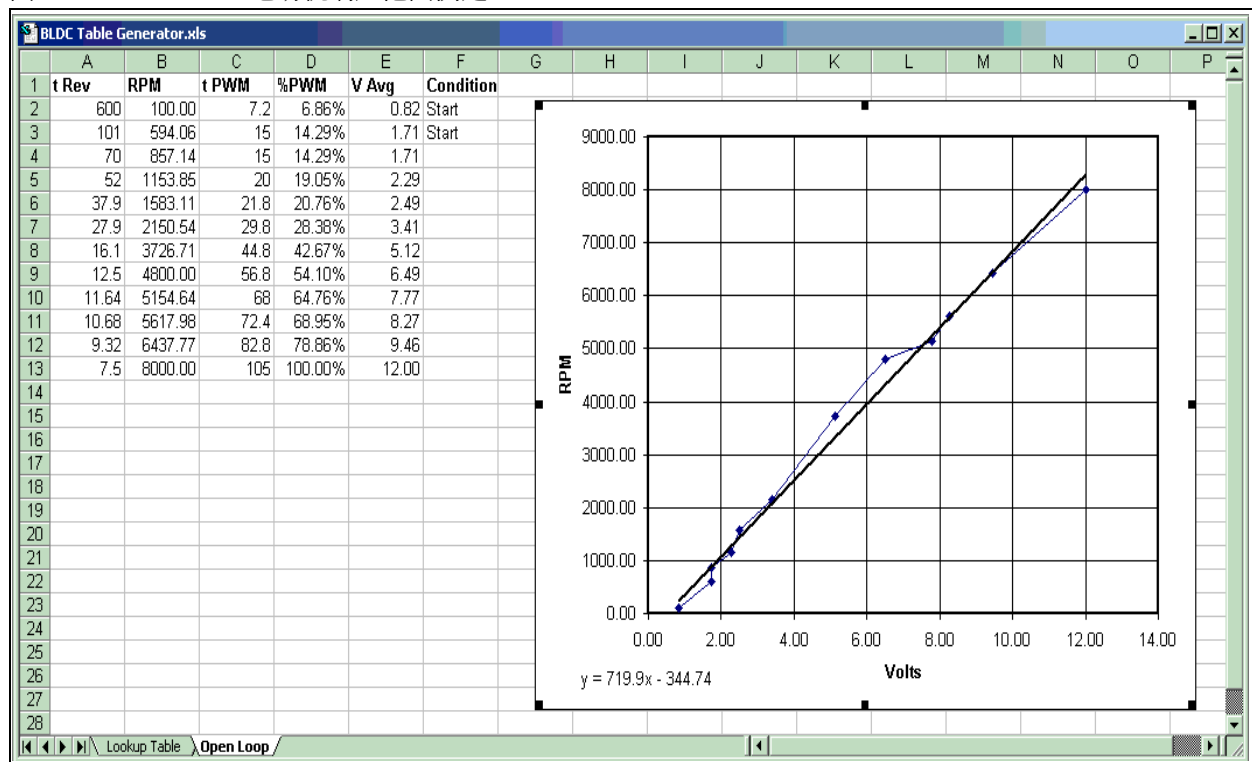
您可以通过以开环模式操作电动机来测量电动机特性，并在几个施加电压下测量电动机电流。然后，您可以在电子表格中（例如Excel）中绘制响应曲线，以确定斜率和偏移数字。最后，将最大rpm插入最大rpm并将其偏移回到表生成器电子表格中，以添加RPM表。

要使用Excel®获得响应偏移，请在电子表格的相邻列中输入电压（左列）和RPM（右列）对。使用图表向导制作X-Y散点图。图表完成后，右键单击响应曲线，然后选择弹出菜单“添加趋势线……”。选项。选择线性回归类型，然后在“选项”选项卡中选中“图表上的显示方程”选项。电子表格的示例如图14所示。

以开环模式操作电动机：

- 将手动阈值编号（Manthresh）设置为0xff。这将防止自动模式接管。
- 以开环模式操作电动机时，首先调整偏移控制，直到电动机开始移动。您可能还需要调整略高于最小值的PWM控件。
- 电动机启动后，您可以增加PWM控件以提高电动机速度。RPM和电压将跟踪，但是您需要经常调整偏移量以优化所选RPM的电压。
- 通过调节最小电流的偏移来优化电压。

图14：电动机响应范围测定



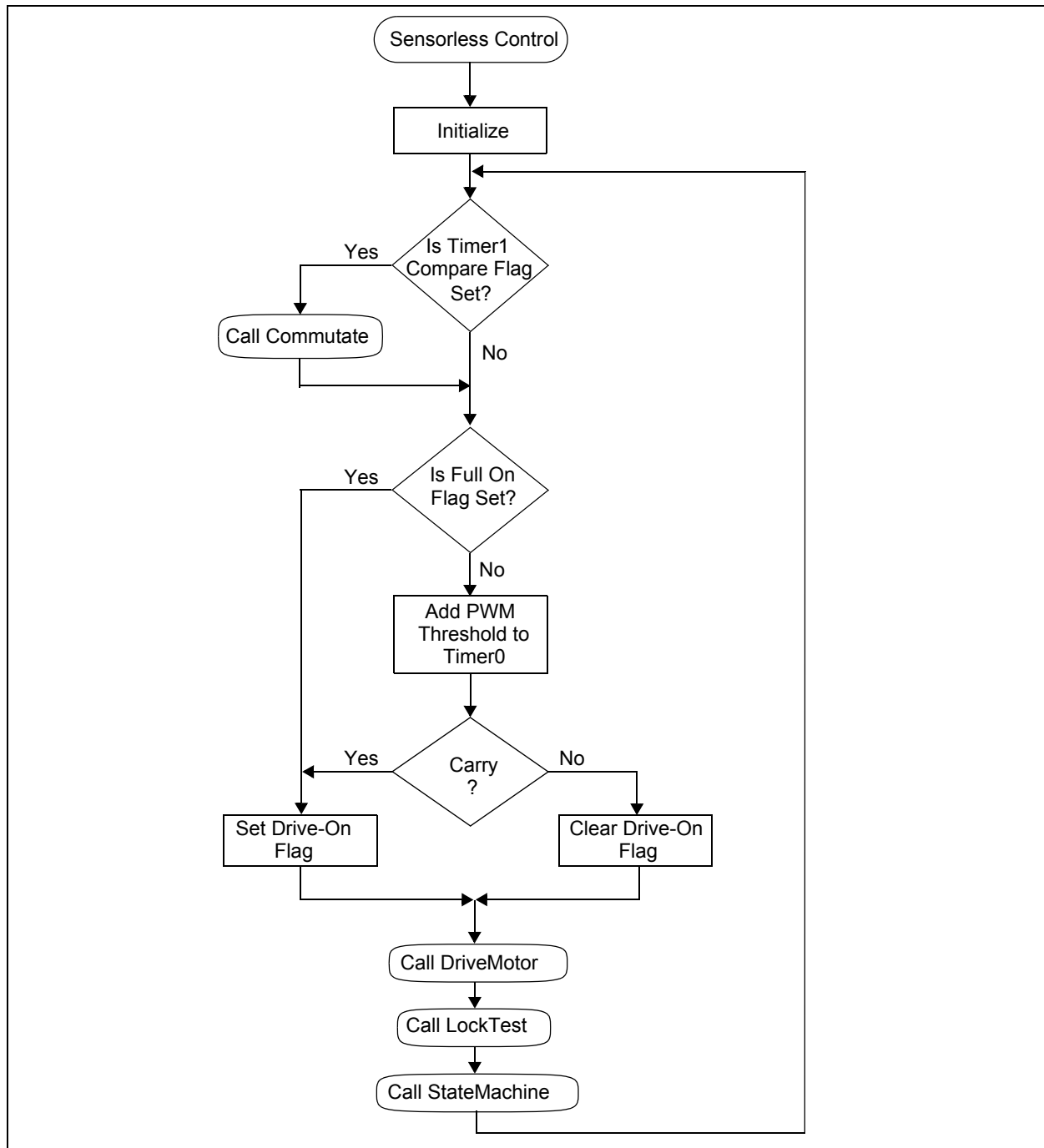
构建无传感器控制代码

在这一点上，我们拥有控制无传感器电机所需的所有部分。我们可以测量BEMF和施加的电压，然后将它们相互比较以确定转子位置。我们可以用PWM改变有效的施加电压，并通过计时换向阶段来控制电动机的速度。某些微不足道的事件必须精确定时。其他微不足道的事件无需互相干扰。必须将ADC从一个源切换到另一个来源，并允许足够的获取时间。某些事件必须以最小的延迟迅速发生。这些包括PWM和通勤。

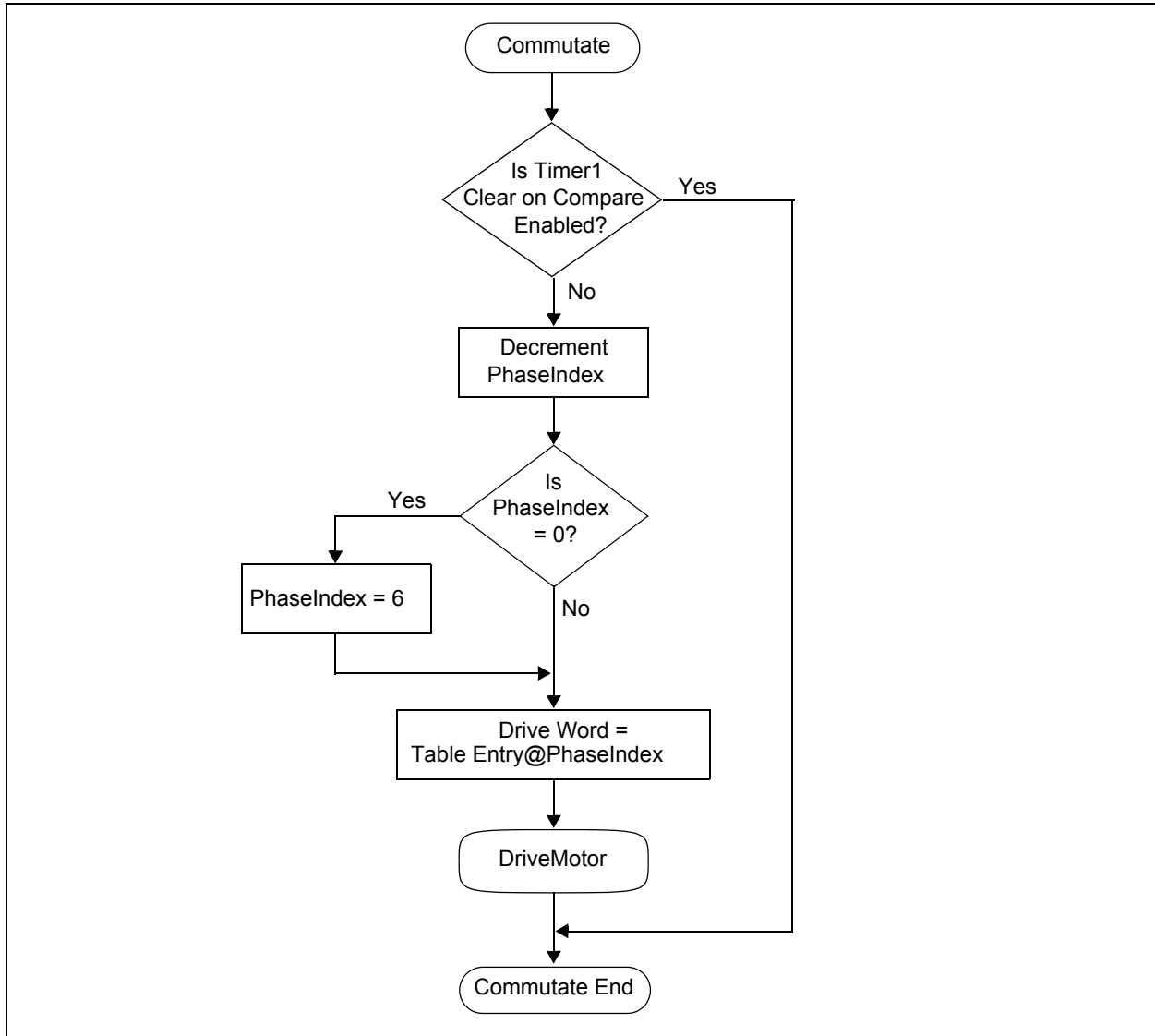
我们可以使用称为状态表的简短主循环来完成所有操作。主循环将处理PWM和换向，状态表将安排读取两个电位计，峰值施加的电压和BEMF电压在连接的电动机端子浮动时两次。图A-1至图A-10，附录A中：“无传感器控制流程图”是传感器电动机控制的结果流程图。代码列表在附录C中：“备件代码”和附录D：“无传感器代码”。

附录A：无传感器控制流程图

图A- 1: MAIN LOOP



图A-2：电动机通勤



图A-3：电动机控制

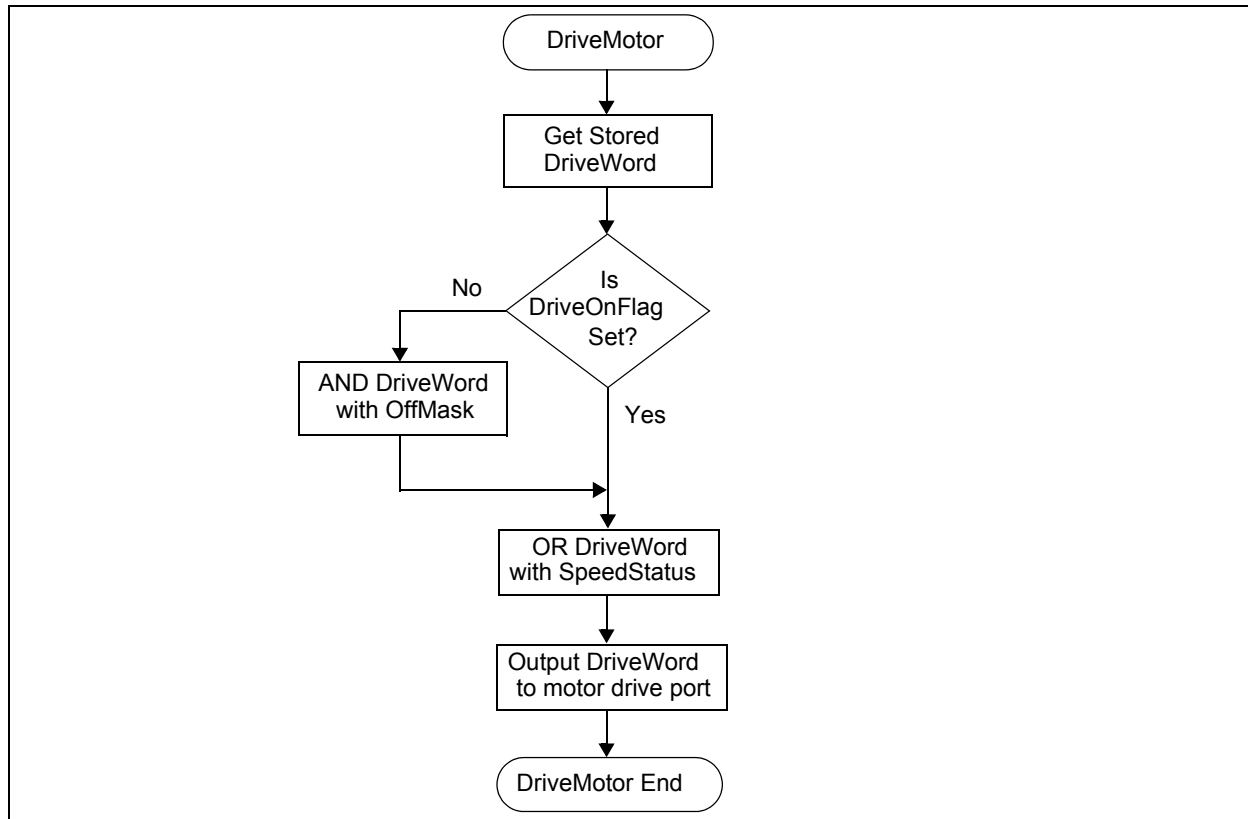
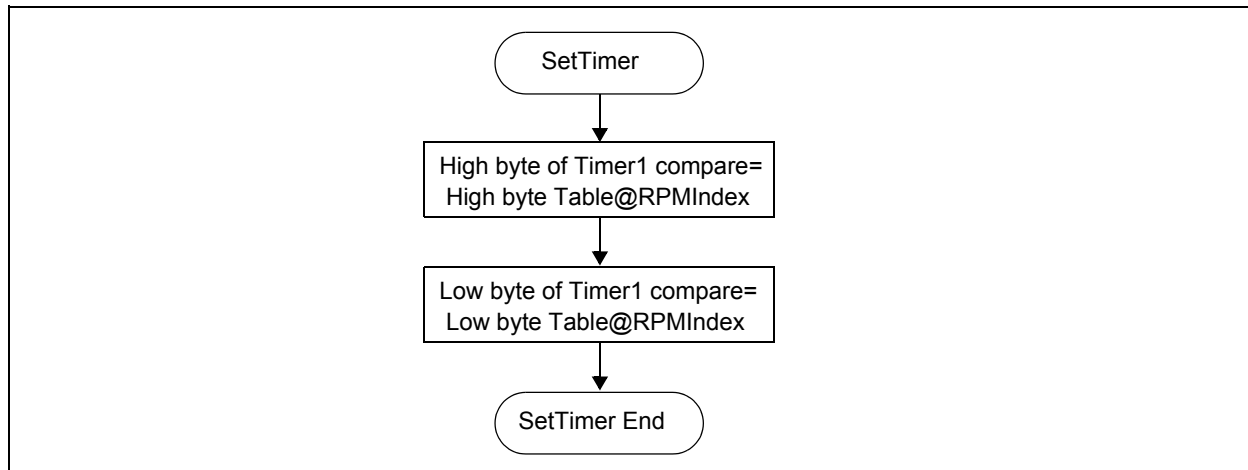


FIGURE A-4: PHASE DRIVE PERIOD



图A-5：电动机速度以换向率锁定

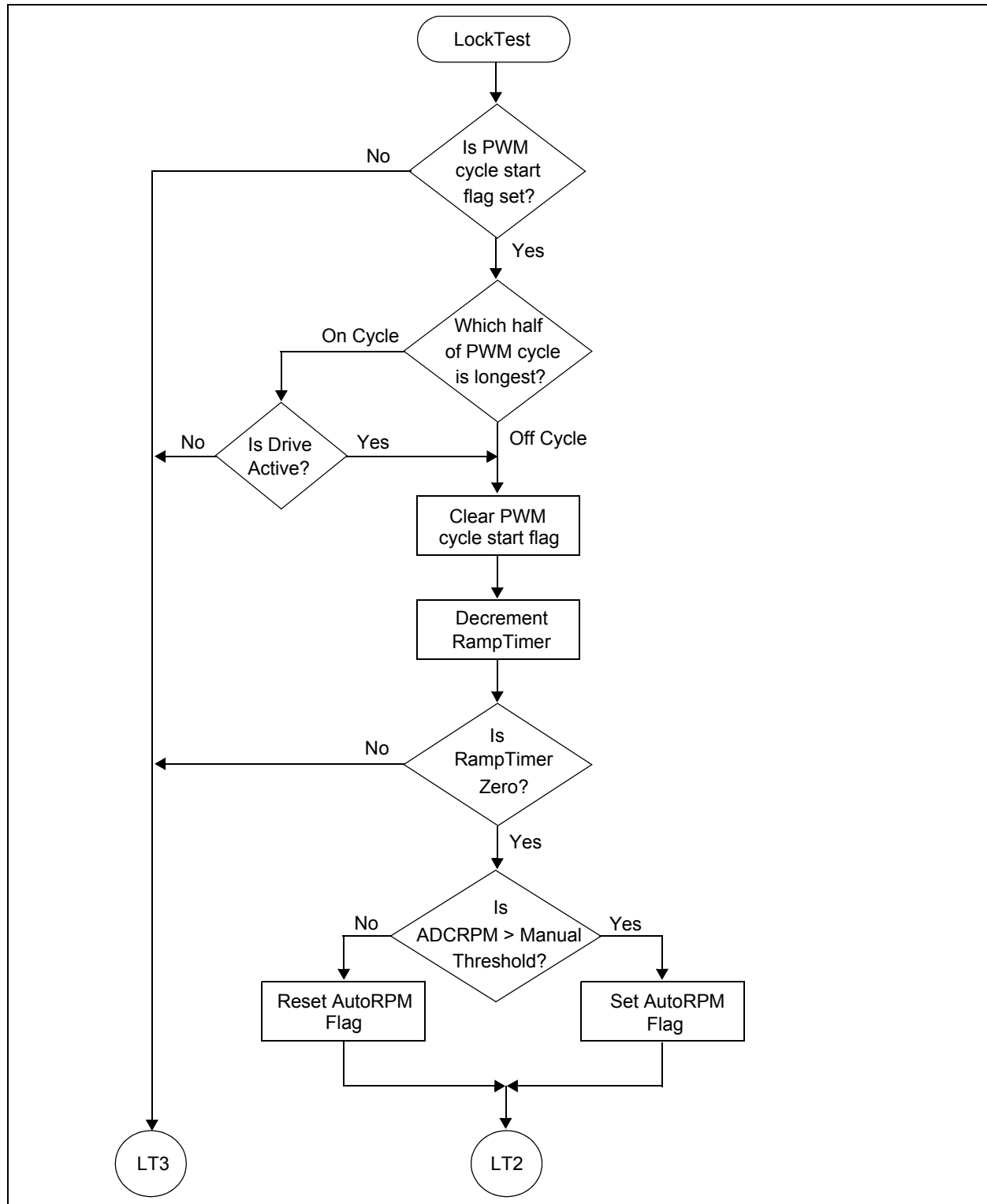


FIGURE A-6: MOTOR SPEED LOCKED WITH COMMUTATION RATE (CONT.)

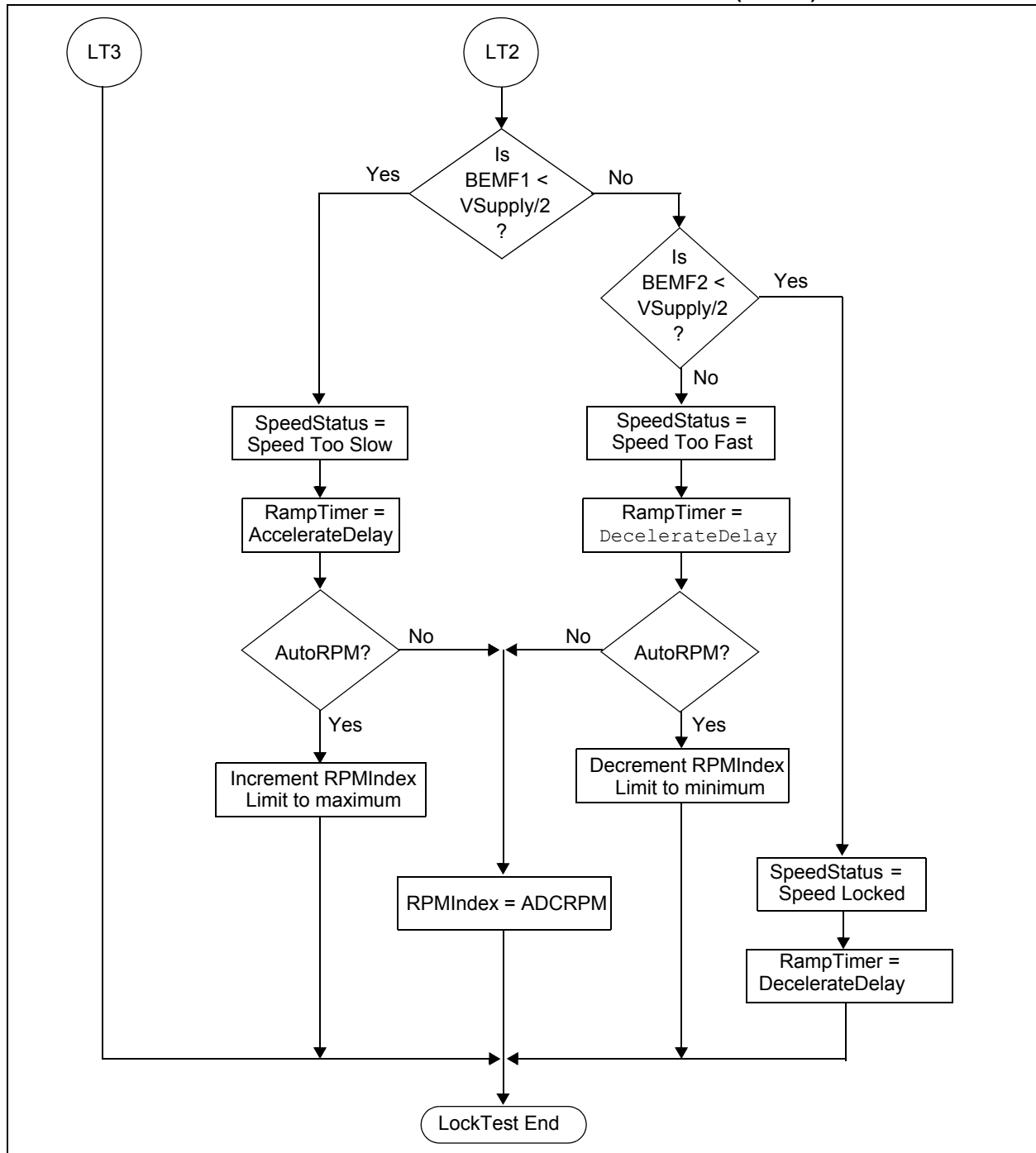


FIGURE A-8: MOTOR CONTROL STATE MACHINE (CONT.)

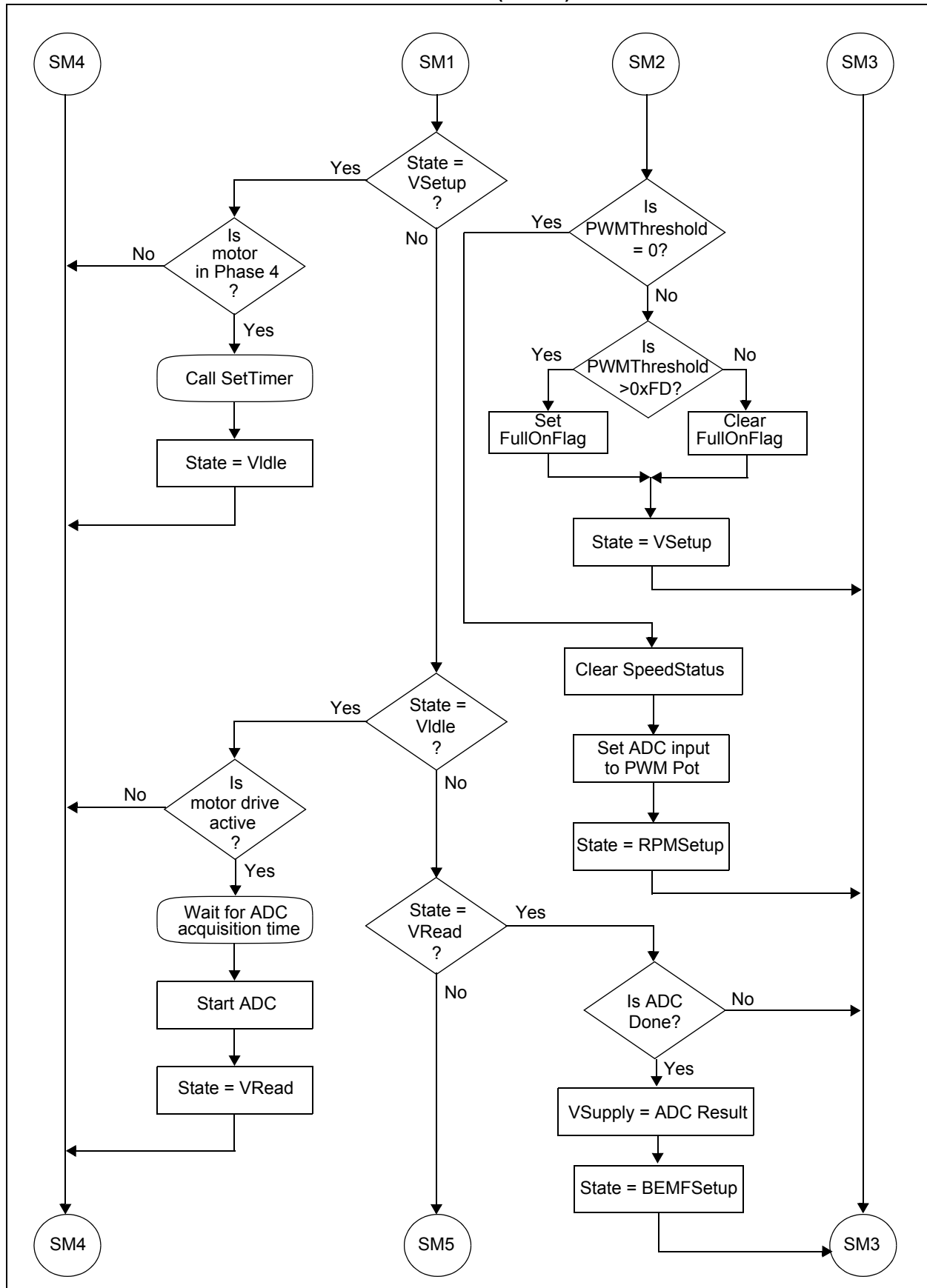
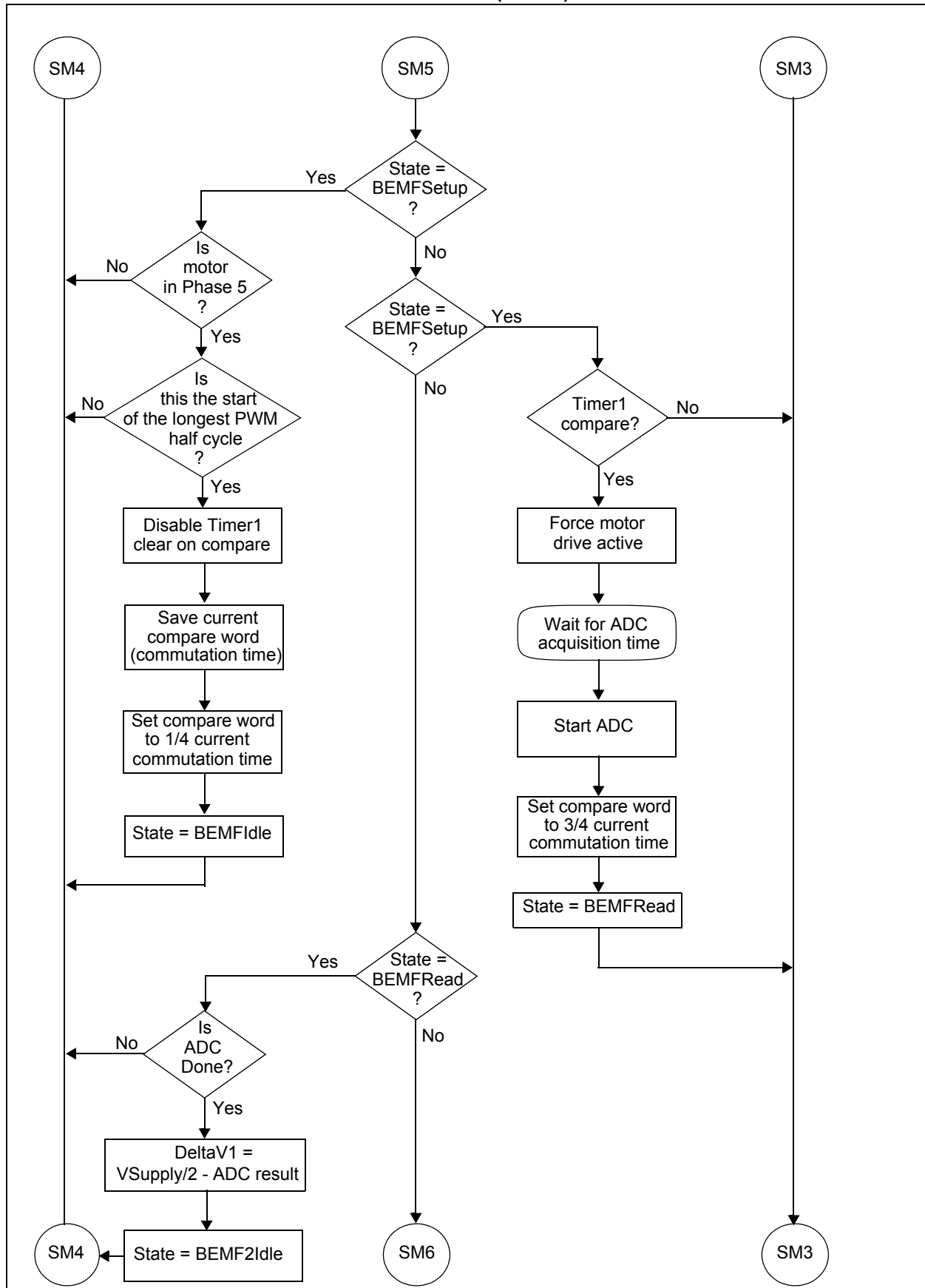
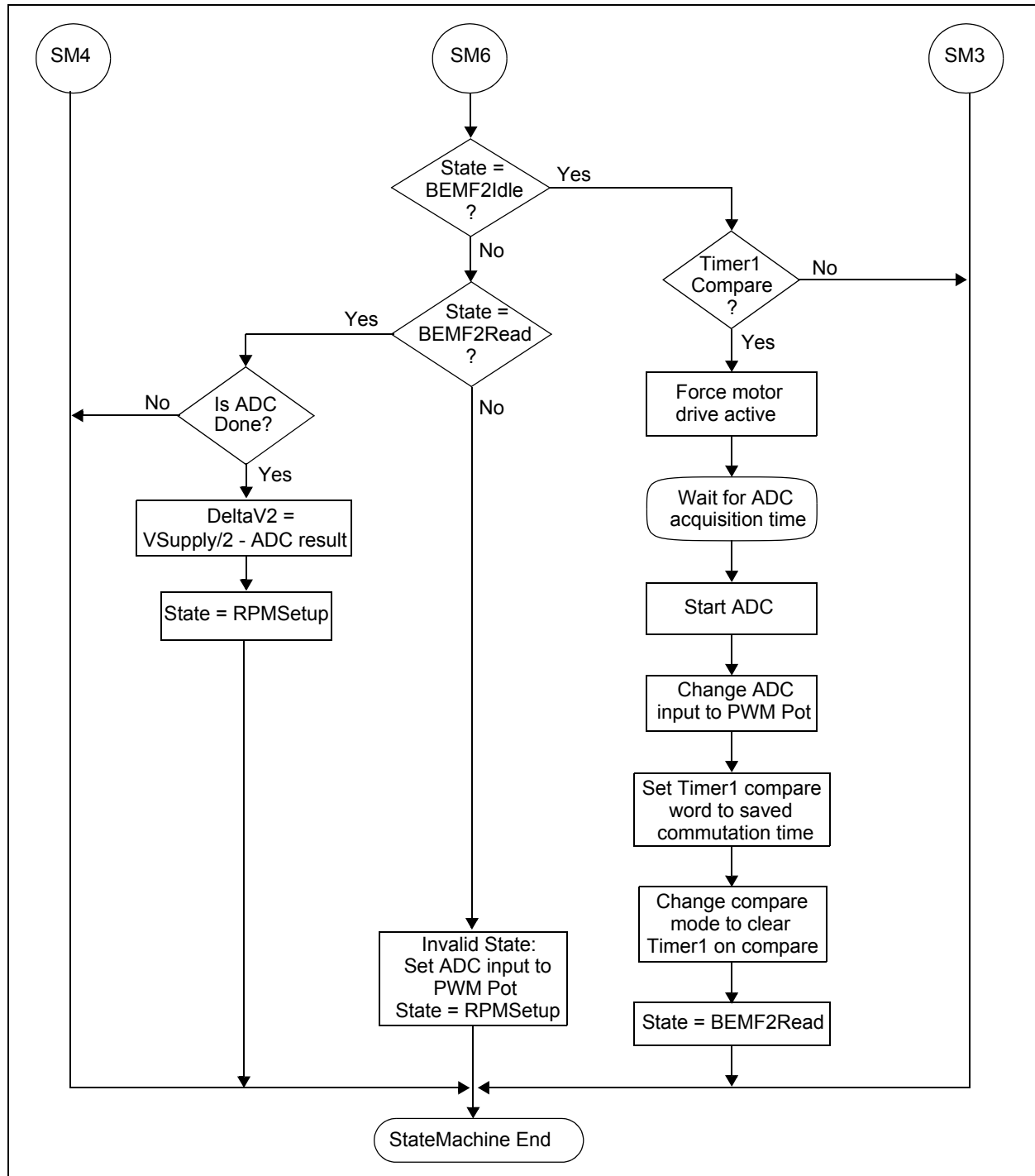


FIGURE A-9: MOTOR CONTROL STATE MACHINE (CONT.)

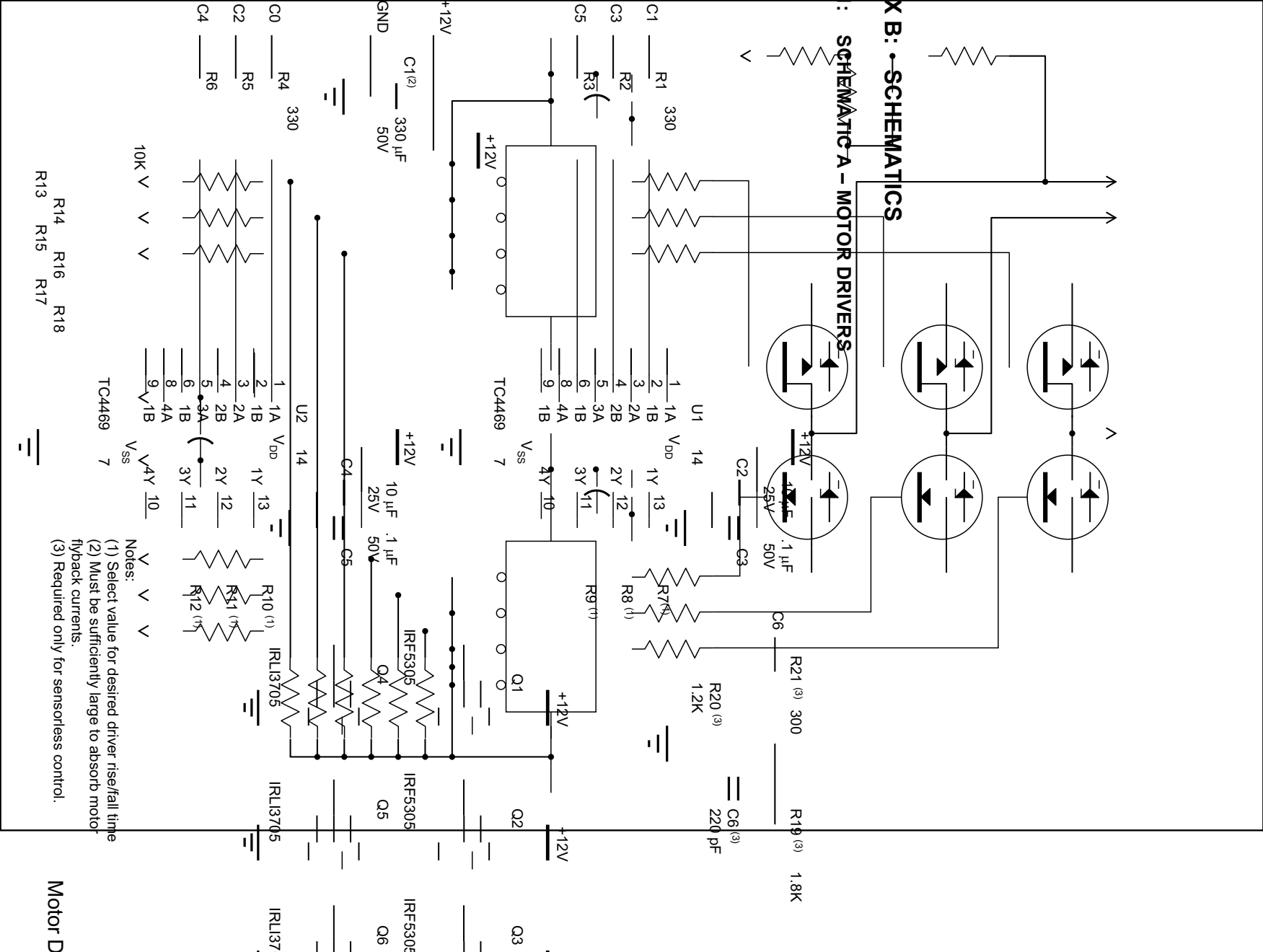


图A-10: 电动机控制状态机 (续)



APPENDIX B: SCHEMATICS

FIGURE B-1: SCHEMATIC A – MOTOR DRIVERS



Motor D

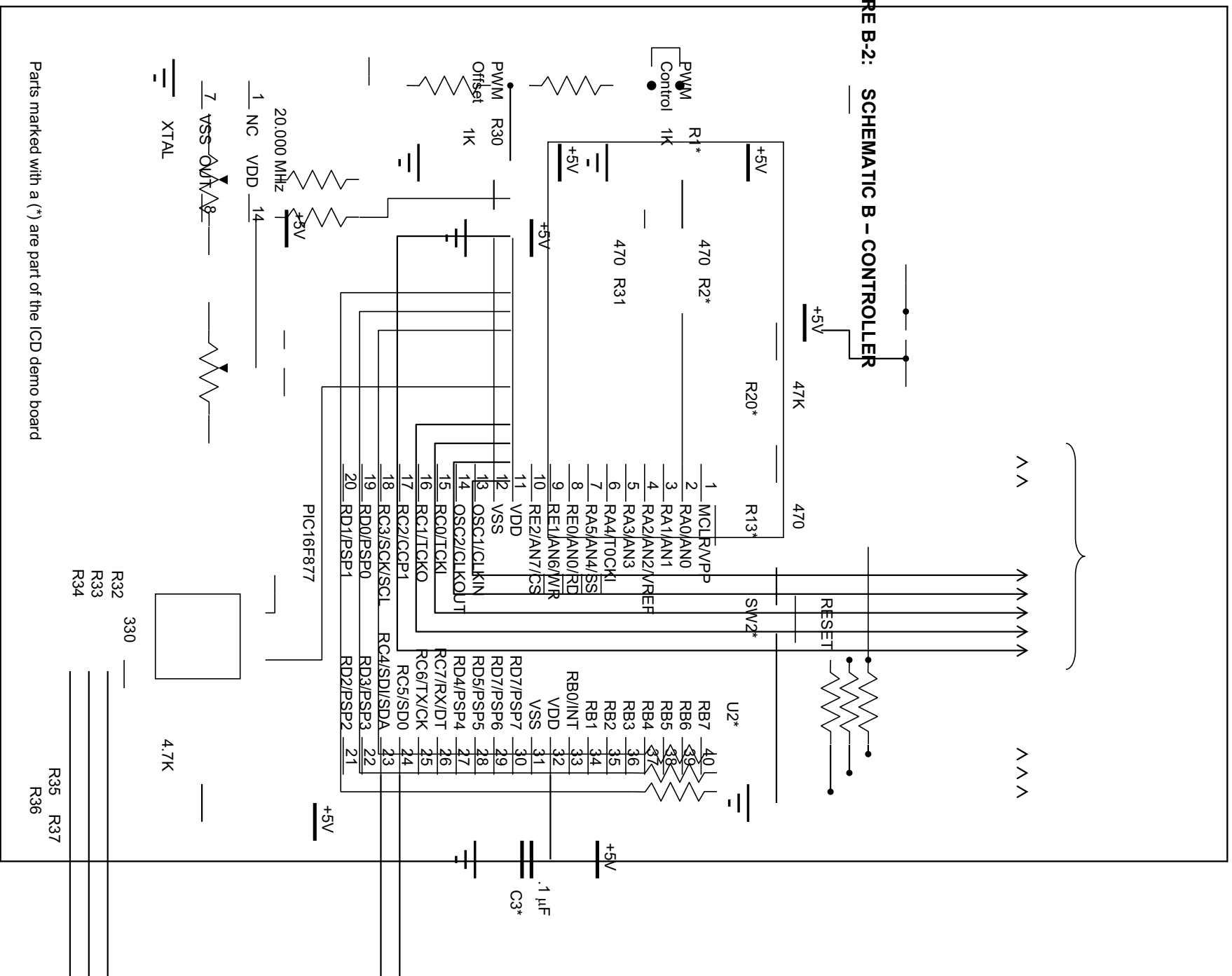


FIGURE B-2: SCHEMATIC B – CONTROLLER

Software License Agreement

The software supplied herewith by Microchip Technology Incorporated (the "Company") is intended and supplied to you, the Company's customer, for use solely and exclusively with products manufactured by the Company.

The software is owned by the Company and/or its supplier, and is protected under applicable copyright laws. All rights are reserved. Any use in violation of the foregoing restrictions may subject the user to criminal sanctions under applicable laws, as well as to civil liability for the breach of the terms and conditions of this license.

THIS SOFTWARE IS PROVIDED IN AN "AS IS" CONDITION. NO WARRANTIES, WHETHER EXPRESS, IMPLIED OR STATUTORY, INCLUDING, BUT NOT LIMITED TO, IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE APPLY TO THIS SOFTWARE. THE COMPANY SHALL NOT, IN ANY CIRCUMSTANCES, BE LIABLE FOR SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES, FOR ANY REASON WHATSOEVER.

附录C：传感代码

```

; *****
; *****;文件名: sensed.asm*;日期: 200
2年2月11日*; 文件版本: 1.0*;*;作者: W.R. Brown*;公司: Microchip Technology Incorporated*;
; *****
; *****;需要: p16f877.inc*;*,*;*, *****
; *****
; *****;注意: 传感的无刷电动机控制主循环使用3位*; 传感器输入作
为驱动单词输出的索引。基于*的PWM; TIMER0控制平均电机电压。确定PWM级别*; PWM级别
是通过ADC读取电位计确定的。*,*, *****
; *****

```

列表P = 16F877;列表指令定义处理器#include < p16f877.inc >;处理器特定变量定义

```

__config _cp_off&_wdt_off&_boden_on&_pwrt_on&_hs_osc&_wrt_enable_off&_lvp_on&_debug_off&_cpdd_off&_cpd_off&
_cpd_off

```

```

; *****;*,*定
义可变存储;*cblock 0x20

```

ADC; PWM阈值是ADC结果LastSensor; 上一次读取电机传
感器数据驱动字; 六位运动驱动数据

ENDC

```

;*****
;*
;* Define I/O
;*

#define    OffMask          B'11010101'
#define    DrivePort        PORTC
#define    DrivePortTris    TRISC
#define    SensorMask       B'00000111'
#define    SensorPort       PORTE
#define    DirectionBit     PORTA,1

;*****

    org      0x000          ; startup vector
    nop                      ; required for ICD operation
    clrf     PCLATH         ; ensure page bits are cleared
    goto     Initialize     ; go to beginning of program

    ORG      0x004          ; interrupt vector location
    retfie                    ; return from interrupt

;*****
;*
;* Initialize I/O ports and peripherals
;*

Initialize
    clrf     DrivePort      ; all drivers off

    banksel  TRISA

; setup I/O
    clrf     DrivePortTris   ; set motor drivers as outputs
    movlw    B'00000011'     ; A/D on RA0, Direction on RA1, Motor sensors on RE<2:0>
    movwf    TRISA          ;
; setup Timer0
    movlw    B'11010000'     ; Timer0: Fosc, 1:2
    movwf    OPTION_REG
; Setup ADC (bank1)
    movlw    B'00001110'     ; ADC left justified, AN0 only
    movwf    ADCON1

    banksel  ADCON0
; setup ADC (bank0)
    movlw    B'11000001'     ; ADC clock from int RC, AN0, ADC on
    movwf    ADCON0

    bsf      ADCON0,GO        ; start ADC
    clrf     LastSensor      ; initialize last sensor reading
    call     Commutate       ; determine present motor position
    clrf     ADC              ; start speed control threshold at zero until first ADC
reading

;*****
;*
;* Main control loop
;*
Loop
    call     ReadADC          ; get the speed control from the ADC
    incfsz   ADC,w           ; if ADC is 0xFF we're at full speed - skip timer add
    goto     PWM              ; add Timer0 to ADC for PWM

    movf     DriveWord,w     ; force on condition
    goto     Drive           ; continue

PWM

```

```

        movf      ADC,w           ; restore ADC reading
        addwf     TMR0,w          ; add it to current Timer0
        movf      DriveWord,w     ; restore commutation drive data
        btfss     STATUS,C        ; test if ADC + Timer0 resulted in carry
        andlw     OffMask         ; no carry - suppress high drivers
Drive
        movwf     DrivePort       ; enable motor drivers
        call      Commutate       ; test for commutation change
        goto      Loop           ; repeat loop

```

ReadAdc; *****
 *****;*如果准备好了, 请阅读速度控制电位计;*并启动下一个读数;*btfsc adcon0, not_done; ADC准备好了吗? 返回;否 - 返回movf adresh, w;获取ADC结果BSF ADCON0, GO; 重新启动ADC MOVWF ADC;保存结果以速度控制阈值返回; ; *****
 *****;*读取传感器输入, 如果感觉到更改, 则从驱动器表中获取;*commutateMovlw sensormask;sensormask;仅保留传感器位和wf Sensorport, W; 获取传感器数据XORWF LastSensor, W;测试运动是否感知BTFSC状态, z; 如果没有更改返回, 则为零; 没有更改 - 回到PWM循环Xorwf Lastsensor, F;用当前的BTFSS方向替换最后一个传感器数据; 测试方向位GOTO FWDCOM;位为零 - 向前换向; 反向换向Movlw高revtable;获取Table movwf pclath的MS字节; 为计算的goto movlw低revtable做准备; 获取table goto com2 fwdcom的LS字节; 向前换向Movlw高fwdtable;获取Table movwf pclath的MS字节; 准备计算的goto movlw低fwdtable; 获取表com2 addwf lastsensor, w;添加传感器偏移BTFSC状态, C; 页面更改表? Incf pclath, f;是的 - 调整MS字节呼叫GetDrive; 从表Movwf Drive Word中获取Drive Word;另存为当前驱动器单词返回getDriveMovWf PCL

```

;*****
;*
;* The drive tables are built based on the following assumptions:
;* 1) There are six drivers in three pairs of two
;* 2) Each driver pair consists of a high side (+V to motor) and low side (motor to ground) drive
;* 3) A 1 in the drive word will turn the corresponding driver on
;* 4) The three driver pairs correspond to the three motor windings: A, B and C
;* 5) Winding A is driven by bits <1> and <0> where <1> is A's high side drive
;* 6) Winding B is driven by bits <3> and <2> where <3> is B's high side drive
;* 7) Winding C is driven by bits <5> and <4> where <5> is C's high side drive
;* 8) Three sensor bits constitute the address offset to the drive table
;* 9) A sensor bit transitions from a 0 to 1 at the moment that the corresponding
;*    winding's high side forward drive begins.
;* 10) Sensor bit <0> corresponds to winding A
;* 11) Sensor bit <1> corresponds to winding B
;* 12) Sensor bit <2> corresponds to winding C
;*
FwdTable
    retlw    B'00000000'        ; invalid
    retlw    B'00010010'        ; phase 6
    retlw    B'00001001'        ; phase 4
    retlw    B'00011000'        ; phase 5
    retlw    B'00100100'        ; phase 2
    retlw    B'00000110'        ; phase 1
    retlw    B'00100001'        ; phase 3
    retlw    B'00000000'        ; invalid

RevTable
    retlw    B'00000000'        ; invalid
    retlw    B'00100001'        ; phase /6
    retlw    B'00000110'        ; phase /4
    retlw    B'00100100'        ; phase /5
    retlw    B'00011000'        ; phase /2
    retlw    B'00001001'        ; phase /1
    retlw    B'00010010'        ; phase /3
    retlw    B'00000000'        ; invalid

    END                        ; directive 'end of program'

```

Software License Agreement

The software supplied herewith by Microchip Technology Incorporated (the "Company") is intended and supplied to you, the Company's customer, for use solely and exclusively with products manufactured by the Company.

The software is owned by the Company and/or its supplier, and is protected under applicable copyright laws. All rights are reserved. Any use in violation of the foregoing restrictions may subject the user to criminal sanctions under applicable laws, as well as to civil liability for the breach of the terms and conditions of this license.

THIS SOFTWARE IS PROVIDED IN AN "AS IS" CONDITION. NO WARRANTIES, WHETHER EXPRESS, IMPLIED OR STATUTORY, INCLUDING, BUT NOT LIMITED TO, IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE APPLY TO THIS SOFTWARE. THE COMPANY SHALL NOT, IN ANY CIRCUMSTANCES, BE LIABLE FOR SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES, FOR ANY REASON WHATSOEVER.

附录D：无传感器代码

```
;*****
;
;   Filename:          snsrless.asm
;   Date:              14 Jan. 2002
;   File Version:      1.0
;
;   Author:            W.R. Brown
;   Company:           Microchip Technology Incorporated
;
;*****
;
;   Files required:    p16f877.inc
;
;*****
;
;   Notes: Sensorless brushless motor control
;
;   Closed loop 3 phase brushless DC motor control.
;   Two potentiometers control operation. One potentiometer (A0)
;   controls PWM (voltage) and RPM (from table). The other
;   potentiometer (A1) provides a PWM offset to the PWM derived
;   from A0. Phase A motor terminal is connected via voltage
;   divider to A3. This is read while the drive is on during
;   phase 4. The result is the peak applied voltage (Vsupply).
;   A3 is also read while the drive is on at two times during
;   phase 5. The result is the BEMF voltage. The BEMF voltage is
;   read at the quarter (t1) and mid (t2) points of the phase 5
;   period. BEMF is compared to VSupply/2. If BEMF is above
;   VSupply/2 at t1 and below VSupply/2w at t2 then no speed
;   adjustment is made. If BEMF is high at both t1 and t2 then
;   the speed is reduced. If BEMF is low at t1 and t2 then the
;   speed is increased.
;*****
;
list P = PIC16F877
include "p16f877.inc"
__CONFIG _CP_OFF & _WRT_ENABLE_OFF & _HS_OSC & _WDT_OFF & _PWRTE_ON & _boden_on

; Acceleration/Deceleration Time = RampRate * 256 * 256 * Timer0Timer0 prescale / fosc

#define AccelDelay      D'100'          ; determines full range acceleration time
#define DecelDelay      D'10'           ; determines full range deceleration time

#define ManThresh       0x3f             ; Manual threshold is the PWM potentiometer
                                         ; reading above which RPM is adjusted automatically
#define AutoThresh      0x100-ManThresh
```

```

OffMask      equ          B'11010101'      ; PWM off kills the high drives
Invalid      equ          B'00000000'      ; invalid
Phase1       equ          B'00100001'      ; phase 1 C high, A low
Phase2       equ          B'00100100'      ; phase 2 C high, B low
Phase3       equ          B'00000110'      ; phase 3 A high, B low
Phase4       equ          B'00010010'      ; phase 4 A high, C low
Phase5       equ          B'00011000'      ; phase 5 B high, C low
Phase6       equ          B'00001001'      ; phase 6 B high, A low

#define       CARRY        STATUS,C
#define       ZERO         STATUS,Z
#define       subw1        sublw

;*****
;*
;* Define I/O Ports
;*

#define       ReadIndicator PORTB,0          ; diagnostic scope trigger for BEMF readings
#define       DrivePort    PORTC            ; motor drive and lock status

;*****
;*
;* Define RAM variables
;*

        CBLOCK 0x20

        STATE              ; Machine state
        PWMThresh          ; PWM threshold
        PhaseIndx          ; Current motor phase index
        Drive              ; Motor drive word
        RPMIndex           ; RPM Index workspace
        ADCRPM             ; ADC RPM value
        ADCOffset          ; Delta offset to ADC PWM threshold
        PresetHi           ; speed control timer compare MS byte
        PresetLo           ; speed control timer compare LS byte
        Flags              ; general purpose flags
        Vsupply            ; Supply voltage ADC reading
        DeltaV1            ; Difference between expected and actual BEMF at T/4
        DeltaV2            ; Difference between expected and actual BEMF at T/2
        CCPSaveH          ; Storage for phase time when finding DeltaV
        CCPSaveL          ; Storage for phase time when finding DeltaV
        CCPT2H             ; Workspace for determining T/2 and T/4
        CCPT2L             ; Workspace for determining T/2 and T/4
        RampTimer          ; Timer0 post scaler for accel/decel ramp rate
        xCount             ; general purpose counter workspace
        Status             ; relative speed indicator status

        ENDC

```

```

;*****
;*
;*      Define Flags
;*

#define DriveOnFlag    Flags,0        ; Flag for invoking drive disable mask when clear
#define AutoRPM        Flags,1        ; RPM timer is adjusted automatically
;                                     Flags,3        ; Undefined
#define FullOnFlag     Flags,4        ; PWM threshold is set to maximum drive
#define Tmr0Ovf        Flags,5        ; Timer0 overflow flag
#define Tmr0Sync       Flags,6        ; Second Timer0 overflow flag
;                                     Flags,7        ; undefined

#define BEMF1Low       DeltaV1,7      ; BEMF1 is low if DeltaV1 is negative
#define BEMF2Low       DeltaV2,7      ; BEMF2 is low if DeltaV2 is negative

;*****
;*
;*      Define State machine states and index numbers
;*

sRPMSetup      equ      D'0'          ; Wait for Phase1, Set ADC GO, RA1->ADC
sRPMRead       equ      sRPMSetup+1   ; Wait for ADC nDONE, Read ADC->RPM
sOffsetSetup   equ      sRPMRead+1    ; Wait for Phase2, Set ADC GO, RA3->ADC
sOffsetRead    equ      sOffsetSetup+1 ; Wait for ADC nDONE, Read ADC->ADCOffset
sVSetup        equ      sOffsetRead+1  ; Wait for Phase4, Drive On, wait 9 uSec, Set ADC GO
sVIdle         equ      sVSetup+1      ; Wait for Drive On, wait Tacq, set ADC GO
sVRead         equ      sVIdle+1       ; Wait for ADC nDONE, Read ADC->Vsupply
sBEMFSetup     equ      sVRead+1       ; Wait for Phase5, set Timer1 compare to half phase time
sBEMFIdle      equ      sBEMFSetup+1   ; Wait for Timer1 compare, Force Drive on and wait 9 uSec,
;                                     ; Set ADC GO, RA0->ADC
sBEMFRead      equ      sBEMFIdle+1    ; Wait for ADC nDONE, Read ADC->Vbemf
sBEMF2Idle     equ      sBEMFRead+1    ; Wait for Timer1 compare, Force Drive on and wait 9 uSec,
;                                     ; Set ADC GO, RA0->ADC
sBEMF2Read     equ      sBEMF2Idle+1   ; Wait for ADC nDONE, Read ADC->Vbemf

;*****
;*
;*      The ADC input is changed depending on the STATE
;*      Each STATE assumes a previous input selection and changes the selection
;*      by XORing the control register with the appropriate ADC input change mask
;*      defined here:
;*

ADC0to1        equ      B'00001000'   ; changes ADCON0<5:3> from 000 to 001
ADC1to3        equ      B'00010000'   ; changes ADCON0<5:3> from 001 to 011
ADC3to0        equ      B'00011000'   ; changes ADCON0<5:3> from 011 to 000

;*****
;***** PROGRAM STARTS HERE *****
;*****

        org      0x000
        nop
        goto     Initialize

        org      0x004
        bsf      Tmr0Ovf        ; Timer0 overflow flag used by accel/decel timer
        bsf      Tmr0Sync       ; Timer0 overflow flag used to synchronize code execution
        bcf      INTCON,T0IF
        retfie                    ;

Initialize
        clrf     PORTC           ; all drivers off
        clrf     PORTB

```

```

        bankssel TRISA
; setup I/O
        clrf      TRISC          ; motor drivers on PORTC
        movlw     B'00001011'    ; A/D on RA0 (PWM), RA1 (Speed) and RA3 (BEMF)
        movwf     TRISA          ;
        movlw     B'11111110'    ; RB0 is locked indicator
        movwf     TRISB
; setup Timer0
        movlw     B'11010000'    ; Timer0: Fosc, 1:2
        movwf     OPTION_REG
        bsf       INTCON,T0IE    ; enable Timer0 interrupts
; Setup ADC
        movlw     B'00000100'    ; ADC left justified, AN0, AN1
        movwf     ADCON1

        bankssel PORTA
        movlw     B'10000001'    ; ADC clk = Fosc/32, AN0, ADC on
        movwf     ADCON0
; setup Timer 1
        movlw     B'00100001'    ; 1:4 prescale, internal clock, timer on
        movwf     T1CON
; setup Timer 1 compare
        movlw     0xFF           ; set compare to maximum count
        movwf     CCP1L1        ; LS compare register
        movwf     CCP1LH        ; MS compare register
        movlw     B'00001011'    ; Timer 1 compare mode, special event - clears timer1
        movwf     CCP1CON

; initialize RAM

        clrf      PWMThresh
        movlw     D'6'
        movwf     PhaseIndx
        clrf      Flags
        clrf      Status          ;
        clrf      STATE          ; LoopIdle->STATE
        bcf       INTCON,T0IF    ; ensure Timer0 overflow flag is cleared
        bsf       INTCON,GIE     ; enable interrupts

MainLoop
;*****
;
;   PWM, Commutation, State machine loop
;
;*****

        btfsc     PIR1,CCP1IF    ; time for phase change?
        call      Commutate      ; yes - change motor drive

PWM
        bsf       DriveOnFlag     ; pre-set flag
        btfsc     FullOnFlag      ; is PWM level at maximum?
        goto      PWM02          ; yes - only commutation is necessary

        movf      PWMThresh,w     ; get PWM threshold
        addwf     TMR0,w          ; compare to Timer0
        btfss     CARRY           ; drive is on if carry is set
        bcf       DriveOnFlag     ; timer has not reached threshold, disable drive

        call      DriveMotor      ; output drive word

PWM02
        call      LockTest
        call      StateMachine    ; service state machine
        goto      MainLoop        ; repeat loop

```

```

StateMachine
    movlw    SMTableEnd-SMTable-1    ; STATE table must have 2^n entries
    andwf    STATE,f                ; limit STATE index to state table
    movlw    high SMTable            ; get high byte of table address
    movwf    PCLATH                  ; prepare for computed goto
    movlw    low SMTable             ; get low byte of table address
    addwf    STATE,w                ; add STATE index to table root
    btfsc    CARRY                   ; test for page change in table
    incf     PCLATH,f               ; page change adjust
    movwf    PCL                    ; jump into table

SMTable
    ; number of STATE table entries MUST be evenly divisible by 2
    goto     RPMSetup               ; Wait for Phase1, Set ADC GO, RA1->ADC, clear Timer0 overflow
    goto     RPMRead                ; Wait for ADC nDONE, Read ADC->RPM
    goto     OffsetSetup            ; Wait for Phase2, Set ADC GO, RA3->ADC
    goto     OffsetRead             ; Wait for ADC nDONE, Read ADC->ADCOffset
    goto     VSetup                 ; Wait for Phase4
    goto     VIdle                  ; Wait for Drive On, wait Tacq, set ADC GO
    goto     VRead                  ; Wait for ADC nDONE, Read ADC->Vsupply
    goto     BEMFSetup              ; Wait for Phase5, set Timer1 compare to half phase time
    goto     BEMFIdle              ; When Timer1 compares force Drive on, Set ADC GO after Tacq,
RA0->ADC
    goto     BEMFRead               ; Wait for ADC nDONE, Read ADC->Vbemf
    goto     BEMF2Idle             ; When Timer1 compares force Drive on, Set ADC GO after Tacq,
RA0->ADC
    goto     BEMF2Read             ; Wait for ADC nDONE, Read ADC->Vbemf

; fill out table with InvalidStates to make number of table entries evenly divisible by 2
    goto     InvalidState          ; invalid state - reset state machine
    goto     InvalidState          ; invalid state - reset state machine
    goto     InvalidState          ; invalid state - reset state machine
    goto     InvalidState          ; invalid state - reset state machine
SMTableEnd

;~~~~~
RPMSetup
    ; Wait for Phase1, Set ADC GO, RA1->ADC, clear Timer0 overflow

    movlw    Phase1                ; compare Phase1 word...
    xorwf    Drive,w               ; ...with current drive word
    btfss    ZERO                   ; ZERO if equal
    return   ; not Phase1 - remain in current STATE

    bsf      ADCON0,GO             ; start ADC
    movlw    ADC0to1              ; prepare to change ADC input
    xorwf    ADCON0,f             ; change from AN0 to AN1
    incf     STATE,f              ; next STATE
    bcf      Tmr0Sync              ; clear Timer0 overflow
    return   ; back to Main Loop

;~~~~~
RPMRead
    ; Wait for ADC nDONE, Read ADC->RPM

    btfsc    ADCON0,GO            ; is ADC conversion finished?
    return   ; no - remain in current STATE

    movf     ADRESH,w             ; get ADC result
    movwf    ADCRPM              ; save in RPM

    incf     STATE,f              ; next STATE
    return   ; back to Main Loop

;~~~~~

```

```

OffsetSetup                                ; Wait for Phase2, Set ADC GO, RA3->ADC

    movlw    Phase2                        ; compare Phase2 word...
    xorwf    Drive,w                      ; ...with current drive word
    btfss    ZERO                          ; ZERO if equal
    return   ; not Phase2 - remain in current STATE

    bsf       ADCON0,GO                    ; start ADC
    movlw     ADC1to3                      ; prepare to change ADC input
    xorwf     ADCON0,f                    ; change from AN1 to AN3
    incf      STATE,f                     ; next STATE
    return    ; back to Main Loop

;~~~~~
OffsetRead                                ; Wait for ADC nDONE, Read ADC->ADCOffset

    btfsc     ADCON0,GO                    ; is ADC conversion finished?
    return    ; no - remain in current STATE

    movf      ADRESH,w                    ; get ADC result
    xorlw     H'80'                      ; complement MSB for +/- offset
    movwf     ADCOffset                  ; save in offset
    addwf     ADCRPM,w                    ; add offset to PWM result
    btfss     ADCOffset,7                 ; is offset a negative number?
    goto      OverflowTest               ; no - test for overflow

    btfss     CARRY                        ; underflow?
    andlw     H'00'                      ; yes - force minimum
    goto      Threshold                  ;

OverflowTest

    btfsc     CARRY                        ; overflow?
    movlw     H'ff'                      ; yes - force maximum

Threshold

    movwf     PWMThresh                  ; PWM threshold is RPM result plus offset
    btfsc     ZERO                        ; is drive off?
    goto      DriveOff                  ; yes - skip voltage measurements

    bcf       FullOnFlag                  ; pre-clear flag in preparation of compare
    sublw     0xFD                        ; full on threshold
    btfss     CARRY                        ; CY = 0 if PWMThresh > FullOn
    bsf       FullOnFlag                  ; set full on flag
    incf      STATE,f                     ; next STATE
    return    ; back to Main Loop

DriveOff

    clrf      Status                      ; clear speed indicators
    movlw     B'11000111'                ; reset ADC input to AN0
    andwf     ADCON0,f                    ;
    clrf      STATE                      ; reset state machine
    return

;~~~~~
VSetup                                    ; Wait for Phase4

    movlw     Phase4                      ; compare Phase4 word...
    xorwf     Drive,w                    ; ...with current Phase drive word
    btfss     ZERO                        ; ZERO if equal
    return    ; not Phase4 - remain in current STATE

    call      SetTimer                    ; set timer value from RPM table
    incf      STATE,f                     ; next STATE
    return    ; back to Main Loop

```

```

;~~~~~
VIdle                                     ; Wait for Drive On, wait Tacq, set ADC GO

    btfss    DriveOnFlag    ; is Drive active?
    return   ; no - remain in current STATE

    call     Tacq           ; motor Drive is active - wait ADC Tacq time
    bsf      ADCON0,GO      ; start ADC
    incf     STATE,f        ; next STATE
    return   ; back to Main Loop

;~~~~~
VRead                                     ; Wait for ADC nDONE, Read ADC->Vsupply

    btfsc     ADCON0,GO      ; is ADC conversion finished?
    return   ; no - remain in current STATE

    movf      ADRESH,w       ; get ADC result
    movwf     Vsupply        ; save as supply voltage
    incf      STATE,f        ; next STATE
    bcf       Tmr0Sync       ; clear Timer0 overflow
    return   ; back to Main Loop

;~~~~~
BEMFSetup                                ; Wait for Phase5, set Timer1 compare to half phase time

    movlw     Phase5         ; compare Phase5 word...
    xorwf     Drive,w        ; ...with current drive word
    btfss     ZERO          ; ZERO if equal
    return    ; not Phase5 - remain in current STATE

    btfss     Tmr0Sync       ; synchronize with Timer0
    return    ;

    btfss     PWMThresh,7    ; if PWMThresh > 0x80 then ON is longer than OFF
    goto      BEMFS1        ; OFF is longer and motor is currently off - compute now

    btfss     DriveOnFlag    ; ON is longer - wait for drive cycle to start
    return    ; not started - wait

BEMFS1

    bcf       CCP1CON,0      ; disable special event on compare
    movf      CCPR1H,w       ; save current capture compare state
    movwf     CCPSaveH      ;
    movwf     CCPT2H         ; save copy in workspace
    movf      CCPR1L,w       ; low byte
    movwf     CCPSaveL      ; save
    movwf     CCPT2L         ; and save copy
    bcf       CARRY          ; pre-clear carry for rotate
    rrf       CCPT2H,f        ; divide phase time by 2
    rrf       CCPT2L,f        ;
    bcf       CARRY          ; pre-clear carry
    rrf       CCPT2H,w        ; divide phase time by another 2
    movwf     CCPR1H         ; first BEMF reading at phase T/4
    rrf       CCPT2L,w        ;
    movwf     CCPR1L         ;

    incf      STATE,f        ; next STATE
    return    ; back to Main Loop

;~~~~~

```

```

BEMFIdle                                     ; When Timer1 compares force Drive on, Set ADC GO after Tacq, RA0-
>ADC

    btfss    PIR1,CCP1IF    ; timer compare?
    return   ; no - remain in current STATE

    bsf      DriveOnFlag    ; force drive on for BEMF reading
    call     DriveMotor     ; activate motor drive
    bsf      ReadIndicator   ; Diagnostic
    call     Tacq           ; wait ADC acquisition time
    bsf      ADCON0,GO       ; start ADC
    bcf      ReadIndicator   ; Diagnostic

; setup to capture BEMF at phase 3/4 T

    movf     CCPT2H,w
    addwf    CCPR1H,f        ; next compare at phase 3/4 T
    movf     CCPT2L,w        ;
    addwf    CCPR1L,f        ; set T/2 lsb
    btfsc    CARRY          ; test for carry into MSb
    incf     CCPR1H,f        ; perform carry
    bcf      PIR1,CCP1IF    ; clear timer compare interrupt flag
    incf     STATE,f        ; next STATE
    return   ; back to Main Loop

;~~~~~
BEMFRead                                     ; Wait for ADC nDONE, Read ADC->Vbemf

    btfsc    ADCON0,GO       ; is ADC conversion finished?
    return   ; no - remain in current STATE

    rrf      Vsupply,w       ; divide supply voltage by 2
    subwf    ADRESH,w        ; Vbemf - Vsupply/2

    movwf    DeltaV1        ; save error voltage
    incf     STATE,f        ; next STATE
    return   ; back to Main Loop

;~~~~~
BEMF2Idle                                    ; When Timer1 compares force Drive on, Set ADC GO after Tacq, RA0-
>ADC

    btfss    PIR1,CCP1IF    ; timer compare?
    return   ; no - remain in current STATE

    bsf      DriveOnFlag    ; force drive on for BEMF reading
    call     DriveMotor     ; activate motor drive
    bsf      ReadIndicator   ; Diagnostic
    call     Tacq           ; wait ADC acquisition time
    bsf      ADCON0,GO       ; start ADC
    bcf      ReadIndicator   ; Diagnostic
    movlw    ADC3to0        ; prepare to change ADC input
    xorwf    ADCON0,f        ; change from AN3 to AN0

; restore Timer1 phase time and special event compare mode

    movf     CCPSaveH,w
    movwf    CCPR1H         ; next compare at phase T
    movf     CCPSaveL,w    ;
    movwf    CCPR1L         ; set T lsb
    bcf      PIR1,CCP1IF    ; clear timer compare interrupt flag
    bsf      CCP1CON,0       ; enable special event on compare
    incf     STATE,f        ; next STATE
    return   ; back to Main Loop

```

```

;~~~~~
BEMF2Read                                ; Wait for ADC nDONE, Read ADC->Vbemf

    btfsc    ADCON0,GO                    ; is ADC conversion finished?
    return   ; no - remain in current STATE

    rrf       Vsupply,w                    ; divide supply voltage by 2
    subwf     ADRESH,w                    ; Vbemf - Vsupply/2

    movwf     DeltaV2                      ; save error voltage

    clrf      STATE                       ; reset state machine to beginning
    return    ; back to Main Loop

;~~~~~
InvalidState                             ; trap for invalid STATE index
    movlw     B'11000111'                 ; reset ADC input to AN0
    andwf     ADCON0,f                    ;
    clrf      STATE                       ;
    return

;

```

tacq; *****

; ADC获取时间的软件延迟; 延迟时间= tosc* (3 + 3*xcount) ;

; *****

movlw d'14; 14等于9个USEC延迟Movwf Xcount; Decfsz Xcount, F; \$ -1;在这里循环直到时间完成返回

锁定

; *****
; T是换向阶段。返回EMF在;在T期间两次浮动运动端子以确定; BEMF的大约零交叉。BEMF
低表示; 测得的BEMF下方 (电源电压) /2。;如果BEMF在1/4 T时低, 则加速。;如果BEMF在1
/4 T时为高, 在3/4 T时为低, 则速度是可以的。;如果BEMF在1/4 T和3/4 T时高, 则减速。;锁定
测试计算与PWM时钟同步。计算是在PWM打开或关闭期间执行的; 时间较长。;; *****

;与Timer0的开始同步测试

```

    btfss     Tmr0Ovf                      ; has Timer0 wrapped around?
    return    ; no - skip lock test

    btfss     PWMThresh,7                  ; if PWMThresh > 0x80 then ON is longer than OFF
    goto      LT05                         ; OFF is longer and motor is currently off - compute now

    btfss     DriveOnFlag                  ; ON is longer - wait for drive cycle to start
    return    ; not started - wait

```

```

LT05
    bcf      Tmr0Ovf      ; clear synchronization flag
    decfsz   RampTimer,f  ; RampTimer controls the acceleration/decel率率
    return

; use lock results to control RPM only if not manual mode

    bsf      AutoRPM      ; preset flag
    movf     ADCRPM,w      ; compare RPM potentiometer...
    addlw    AutoThresh    ; ...to the auto control threshold
    btfss    CARRY        ; CARRY is set if RPM is > auto threshold
    bcf      AutoRPM      ; not in auto range - reset flag

    btfss    BEMF1Low     ; is first BEMF below Supply/2
    goto     LT20         ; no - test second BEMF

LT10
; accelerate if BEMF at 1/4 T is below Supply/2

    movlw    B'10000000'   ; indicate lock test results
    movwf    Status        ; status is OR'd with drive word later
    movlw    AccelDelay    ; set the timer for acceleration delay
    movwf    RampTimer     ;

    btfss    AutoRPM      ; is RPM in auto range?
    goto     ManControl    ; no - skip RPM adjustment

    incfsz   RPMIndex,f    ; increment the RPM table index
    return                    ; return if Index didn't wrap around

    decf     RPMIndex,f    ; top limit is 0xFF
    return

LT20
    btfsc    BEMF2Low     ; BEMF1 was high...
    goto     ShowLocked    ; ... and BEMF2 is low - show locked

; decelerate if BEMF at 3/4 T is above Supply/2

    movlw    B'01000000'   ; indicate lock test results
    movwf    Status        ; status is OR'd with drive word later
    movlw    DecelDelay    ; set the timer for deceleration delay
    movwf    RampTimer     ;

    btfss    AutoRPM      ; is RPM in auto range?
    goto     ManControl    ; no - skip RPM adjustment

    decfsz   RPMIndex,f    ; set next lower RPM table index
    return                    ; return if index didn't wrap around

    incf     RPMIndex,f    ; bottom limit is 0x01
    return

ShowLocked
    movlw    B'11000000'   ; indicate lock test results
    movwf    Status        ; status is OR'd with drive word later
    movlw    DecelDelay    ; set the timer for deceleration delay
    movwf    RampTimer     ;

    btfsc    AutoRPM      ; was RPM set automatically?
    return                    ; yes - we're done

```

```

ManControl
    movf    ADCRPM,w        ; get RPM potentiometer reading...
    movwf   RPMIndex        ; ...and set table index directly
    return

Commutate
;*****
;
;   Commutation is triggered by PIR1<CCP1IF> flag.
;   This flag is set when timer1 equals the compare register.
;   When BEMF measurement is active the compare time is not
;   cleared automatically (special event trigger is off).
;   Ignore the PIR1<CCP1IF> flag when special trigger is off
;   because the flag is for BEMF measurement.
;   If BEMF measurement is not active then decrement phase table
;   index and get the drive word from the table. Save the
;   drive word in a global variable and output to motor drivers.
;*****

    btfss   CCP1CON,0        ; is special event on compare enabled?
    return                                ; no - this is a BEMF measurement, let state machine handle this

    bcf     PIR1,CCP1IF      ; clear interrupt flag

    movlw   high OnTable     ; set upper program counter bits
    movwf   PCLATH
    decfsz  PhaseIndx,w      ; decrement to next phase
    goto    $+2              ; skip reset if not zero
    movlw   D'6'             ; phase counts 6 to 1
    movwf   PhaseIndx        ; save the phase index
    addlw   LOW OnTable
    btfsc   CARRY            ; test for possible page boundary
    incf    PCLATH,f         ; page boundary adjust
    call    GetDrive
    movwf   Drive            ; save motor drive word

DriveMotor
    movf    Drive,w          ; restore motor drive word
    btfss   DriveOnFlag      ; test drive enable flag
    andlw   OffMask          ; kill high drive if PWM is off
    iorwf   Status,w         ; show speed indicators
    movwf   DrivePort        ; output to motor drivers
    return

GetDrive
    movwf   PCL              ; computed goto

OnTable
    retlw   Invalid
    retlw   Phase6
    retlw   Phase5
    retlw   Phase4
    retlw   Phase3
    retlw   Phase2
    retlw   Phase1
    retlw   Invalid

```

Settimer

```

;*****
;
;      This sets the CCP module compare registers for timer 1.
;      The motor phase period is the time it takes timer 1
;      to count from 0 to the compare value. The CCP module
;      is configured to clear timer 1 when the compare occurs.
;      Get the timer1 compare variable from two lookup tables, one
;      for the compare high byte and the other for the low byte.
;
;*****

    call    SetTimerHigh
    movwf   CCPR1H          ; Timer1 High byte preset
    call    SetTimerLow
    movwf   CCPR1L          ; Timer1 Low byte preset
    return

SetTimerHigh
    movlw   high T1HighTable ; lookup preset values
    movwf   PCLATH           ; high bytes first
    movlw   low T1HighTable  ;
    addwf   RPMIndex,w       ; add table index
    btfsc   STATUS,C         ; test for table page crossing
    incf    PCLATH,f         ;
    movwf   PCL              ; lookup - result returned in W

SetTimerLow
    movlw   high T1LowTable   ; repeat for lower byte
    movwf   PCLATH           ;
    movlw   low T1LowTable    ;
    addwf   RPMIndex,w       ; add table index
    btfsc   STATUS,C         ; test for table page crossing
    incf    PCLATH,f         ;
    movwf   PCL              ; lookup - result returned in W

#include "bldcspd4.inc"

    结尾

```

请注意Microchip设备上代码保护功能的以下详细信息：

- 微芯片产品符合其特定的微芯片数据表中包含的规范。
- Microchip认为，当以预期的方式和正常条件下使用时，其产品家族是当今市场上最安全的家族之一。
- 有不诚实的方法，可能是用于违反代码保护功能的非法方法。据我们所知，所有这些方法都需要以微芯片数据表中包含的操作规格之外的方式使用微芯片产品。这样做的人最有可能盗窃知识产权。
- Microchip愿意与关注其代码完整性的客户合作。
- 微芯片和任何其他半导体制造商都无法保证其代码的安全性。代码保护并不意味着我们保证产品为“牢不可破”。

代码保护正在不断发展。我们在Microchip致力于不断提高产品的代码保护功能。试图打破微芯片的代码保护功能的尝试可能违反了《数字千年版权法》。如果此类行为允许未经授权访问您的软件或其他受版权保护的工作，则您可能有权根据该法案提起救济。

本出版物中包含的有关设备应用程序之类的信息仅出于您的便利而提供，并且可能被更新所取代。您有责任确保您的申请符合您的规格。Microchip不做任何明示或暗示，书面或口头，法定或其他与信息相关的陈述或保证，包括但不限于其条件，质量，绩效，适用性或适合目的的状况。Microchip不承担此信息及其使用而产生的所有责任。在生命支持和/或安全申请中使用微芯片设备完全符合买方的风险，买方同意捍卫，赔偿和持有无害的微芯片，从任何损害，索赔，诉讼，诉讼或费用中造成的使用。除非另有说明，否则在任何微芯片知识产权下，任何授权或其他方面都不会传递许可。

商标

The Microchip name and logo, the Microchip logo, Adaptec, AnyRate, AVR, AVR logo, AVR Freaks, BesTime, BitCloud, chipKIT, chipKIT logo, CryptoMemory, CryptoRF, dsPIC, FlashFlex, flexPWR, HELDO, IGLOO, JukeBlox, KeeLoq, Klear, LANCheck, LinkMD, maXStylus, maXTouch, MediaLB, megaAVR, Microsemi, Microsemi logo, MOST, MOST logo, MPLAB, OptoLyzer, PackTime, PIC, picoPower, PICSTART, PIC32 logo, PolarFire, Prochip Designer, QTouch, SAM-BA, SenGenuity, SpyNIC, SST, SST Logo, SuperFlash, Symmetricom, SyncServer, Tachyon, Temptrackr, Timeource, Tinyavr, Uni/O, Vectron和Xmega是美国和其他国家/地区的Microchip Technology的注册商标。

APT, ClockWorks, The Embedded Control Solutions Company, EtherSynch, FlashTec, Hyper Speed Control, HyperLight Load, IntelliMOS, Libero, motorBench, mTouch, Powermite 3, Precision Edge, ProASIC, ProASIC Plus, ProASIC Plus logo, Quiet-Wire, SmartFusion, SyncWorld, Temux, TimeCesium, TimeHub, TimePictra, TimeProvider, Vite, WinPath, ZL和美国的Microchip Technology的注册商标。

Adjacent Key Suppression, AKS, Analog-for-the-Digital Age, AnyCapacitor, AnyIn, AnyOut, BlueSky, BodyCom, CodeGuard, CryptoAuthentication, CryptoAutomotive, CryptoCompanion, CryptoController, dsPICDEM, dsPICDEM.net, Dynamic Average Matching, DAM, ECAN, EtherGREEN, In-Circuit Serial Programming, ICSP, INICNET, 片间连通性, jitterblocker,

KlearNet, KlearNet logo, memBrain, Mindi, MiWi, MPASM, MPF, MPLAB Certified logo, MPLIB, MPLINK, MultiTRAK, NetDetach, Omniscient Code Generation, PICDEM, PICDEM.net, PICkit, PICtail, PowerSmart, PureSilicon, QMatrix, REAL ICE, Ripple Blocker, SAM-ICE, Serial Quad I/O, Smart-I.S., SQI, Superswitcher, Superswitcher II, Total Endurance, TSHARC, USBCHECK, VARISENSE, VIVESSENS, VIEWSPAN, WIPERLOCK, WIREless DNA和Zena是美国和其他国家/地区的Microchip Technology商标。

SQTP是美国融合的Microchip Technology的服务标记。

Adaptec徽标，按需频率，硅存储技术和SYMMCOM是其他国家的Microchip Technology Inc.的注册商标。Gestic是Microchip Technology Inc.的子公司Microchip Technology GmbH & Co. KG的注册商标。

本文提到的所有其他商标都是其各自公司的财产。

©2002-2019, Microchip Technology Incorporated, 保留所有权利。

ISBN: 978-1-5224-4689-7

For information regarding Microchip's Quality Management Systems, please visit www.microchip.com/quality.

Worldwide Sales and Service

AMERICAS

Corporate Office
2355 West Chandler Blvd.
Chandler, AZ 85224-6199
Tel: 480-792-7200
Fax: 480-792-7277
Technical Support:
<http://www.microchip.com/support>
Web Address:
www.microchip.com

Atlanta
Duluth, GA
Tel: 678-957-9614
Fax: 678-957-1455

Austin, TX
Tel: 512-257-3370

Boston
Westborough, MA
Tel: 774-760-0087
Fax: 774-760-0088

Chicago
Itasca, IL
Tel: 630-285-0071
Fax: 630-285-0075

Dallas
Addison, TX
Tel: 972-818-7423
Fax: 972-818-2924

Detroit
Novi, MI
Tel: 248-848-4000

Houston, TX
Tel: 281-894-5983

Indianapolis
Noblesville, IN
Tel: 317-773-8323
Fax: 317-773-5453
Tel: 317-536-2380

Los Angeles
Mission Viejo, CA
Tel: 949-462-9523
Fax: 949-462-9608
Tel: 951-273-7800

Raleigh, NC
Tel: 919-844-7510

New York, NY
Tel: 631-435-6000

San Jose, CA
Tel: 408-735-9110
Tel: 408-436-4270

Canada - Toronto
Tel: 905-695-1980
Fax: 905-695-2078

ASIA/PACIFIC

Australia - Sydney
Tel: 61-2-9868-6733

China - Beijing
Tel: 86-10-8569-7000

China - Chengdu
Tel: 86-28-8665-5511

China - Chongqing
Tel: 86-23-8980-9588

China - Dongguan
Tel: 86-769-8702-9880

China - Guangzhou
Tel: 86-20-8755-8029

China - Hangzhou
Tel: 86-571-8792-8115

China - Hong Kong SAR
Tel: 852-2943-5100

China - Nanjing
Tel: 86-25-8473-2460

China - Qingdao
Tel: 86-532-8502-7355

China - Shanghai
Tel: 86-21-3326-8000

China - Shenyang
Tel: 86-24-2334-2829

China - Shenzhen
Tel: 86-755-8864-2200

China - Suzhou
Tel: 86-186-6233-1526

China - Wuhan
Tel: 86-27-5980-5300

China - Xian
Tel: 86-29-8833-7252

China - Xiamen
Tel: 86-592-2388138

China - Zhuhai
Tel: 86-756-3210040

ASIA/PACIFIC

India - Bangalore
Tel: 91-80-3090-4444

India - New Delhi
Tel: 91-11-4160-8631

India - Pune
Tel: 91-20-4121-0141

Japan - Osaka
Tel: 81-6-6152-7160

Japan - Tokyo
Tel: 81-3-6880-3770

Korea - Daegu
Tel: 82-53-744-4301

Korea - Seoul
Tel: 82-2-554-7200

Malaysia - Kuala Lumpur
Tel: 60-3-7651-7906

Malaysia - Penang
Tel: 60-4-227-8870

Philippines - Manila
Tel: 63-2-634-9065

Singapore
Tel: 65-6334-8870

Taiwan - Hsin Chu
Tel: 886-3-577-8366

Taiwan - Kaohsiung
Tel: 886-7-213-7830

Taiwan - Taipei
Tel: 886-2-2508-8600

Thailand - Bangkok
Tel: 66-2-694-1351

Vietnam - Ho Chi Minh
Tel: 84-28-5448-2100

EUROPE

Austria - Wels
Tel: 43-7242-2244-39
Fax: 43-7242-2244-393

Denmark - Copenhagen
Tel: 45-4450-2828
Fax: 45-4485-2829

Finland - Espoo
Tel: 358-9-4520-820

France - Paris
Tel: 33-1-69-53-63-20
Fax: 33-1-69-30-90-79

Germany - Garching
Tel: 49-8931-9700

Germany - Haan
Tel: 49-2129-3766400

Germany - Heilbronn
Tel: 49-7131-72400

Germany - Karlsruhe
Tel: 49-721-625370

Germany - Munich
Tel: 49-89-627-144-0
Fax: 49-89-627-144-44

Germany - Rosenheim
Tel: 49-8031-354-560

Israel - Ra'anana
Tel: 972-9-744-7705

Italy - Milan
Tel: 39-0331-742611
Fax: 39-0331-466781

Italy - Padova
Tel: 39-049-7625286

Netherlands - Drunen
Tel: 31-416-690399
Fax: 31-416-690340

Norway - Trondheim
Tel: 47-7288-4388

Poland - Warsaw
Tel: 48-22-3325737

Romania - Bucharest
Tel: 40-21-407-87-50

Spain - Madrid
Tel: 34-91-708-08-90
Fax: 34-91-708-08-91

Sweden - Gothenberg
Tel: 46-31-704-60-40

Sweden - Stockholm
Tel: 46-8-5090-4654

UK - Wokingham
Tel: 44-118-921-5800
Fax: 44-118-921-5820